
L2G-Net: Local to Global Spectral Graph Neural Networks via Cauchy Factorizations

Samuel Fernández-Menduiña¹ Eduardo Pavez¹ Antonio Ortega¹

Abstract

Despite their theoretical advantages, spectral methods based on the graph Fourier transform (GFT) are seldom used in graph neural networks (GNNs) due to the cost of computing the eigenbasis and the lack of vertex-domain locality in the resulting representations. As a result, most GNNs rely on local approximations such as polynomial Laplacian filters or message passing, which limit their ability to model long-range dependencies. In this paper, we introduce an exact factorization of the GFT into operators acting on subgraphs, which are then combined via a sequence of Cauchy matrices. Building on this factorization, we propose a new class of spectral GNNs, termed L2G-Net (Local to Global Net). Unlike existing spectral methods, which are either fully global (when using the GFT) or local (when using polynomial filters), L2G-Net operates by processing the spectral representations of subgraphs and then combining them via structured matrices. Our algorithm avoids full eigendecompositions, exploiting graph topology to construct the factorization with quadratic complexity in the number of nodes, scaled by the maximum cut size between subgraphs. Experiments stressing long-range dependencies on large graphs show that L2G-Net scales to regimes out of reach for the standard GFT, and is competitive with state-of-the-art methods with orders of magnitude fewer learnable parameters.

1. Introduction

Spectral graph neural networks (Bruna et al., 2014) process graph signals by projecting them onto the eigenbasis of the

graph Laplacian. This operation, also known as the graph Fourier transform (GFT) (Ortega et al., 2018), is analogous to the Fourier transform in signal processing in regular domains, yielding a representation that captures the global structure of the graph (Chung, 1997).

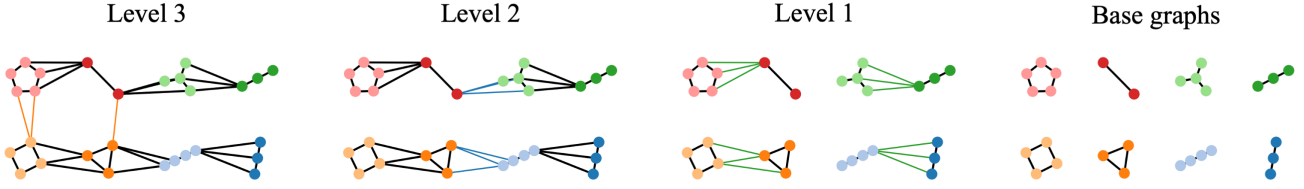
Despite using well-defined graph signal frequencies, GFT-based GNNs are far less used in practice than polynomial approximations (Defferrard et al., 2016; Levie et al., 2018; He et al., 2021) or message-passing neural networks (MPNNs) (Kipf and Welling, 2017). Two limitations drive this lack of adoption: the cubic complexity of computing the GFT makes it impractical for large graphs (Bronstein et al., 2017), and GFT-domain operations are global, i.e., modifying a single spectral coefficient affects all nodes simultaneously (Ortega et al., 2018). While globality is well-suited to capture long-range interactions (Dwivedi et al., 2022b), the GFT lacks the local inductive biases that benefit short-range graph learning tasks, which dominate many of the existing benchmarks (Alon and Yahav, 2021).

Polynomial approximations and MPNNs replace global GFT operations with repeated products with a sparse graph operator (e.g., adjacency or Laplacian) (Kipf and Welling, 2017) or a learned polynomial of the operator (Defferrard et al., 2016). These approaches are computationally efficient and induce a k -hop locality bias, whose alignment with widely-used homophilic benchmarks partially explains their empirical adoption (Hariri et al., 2026). However, modeling long-range dependencies with local operators requires multiple message-passing steps, often leading to optimization instabilities (Arroyo et al., 2026) such as oversquashing (Alon and Yahav, 2021). Replacing the Laplacian with a polynomial of the Laplacian might alleviate these issues, but often at the cost of restrictive parameter constraints (Hariri et al., 2026). Combining global and local information via self-attention (Deng et al., 2024) often sacrifices parameter efficiency and graph-domain interpretability (cf. Appendix B).

In this paper, we introduce a new class of spectral GNNs that builds *global* spectral representations by aggregating *local* spectral coefficients (Figure 1). We show that any GFT can be *exactly factorized* into a sequence of localized transformations: given a graph partition (Figure 1 (a)), the

¹University of Southern California. Correspondence to: Samuel Fernández-Menduiña <samuelf9@usc.edu>.

(a) A hierarchical graph



(b) Spectral processing in L2G-Net

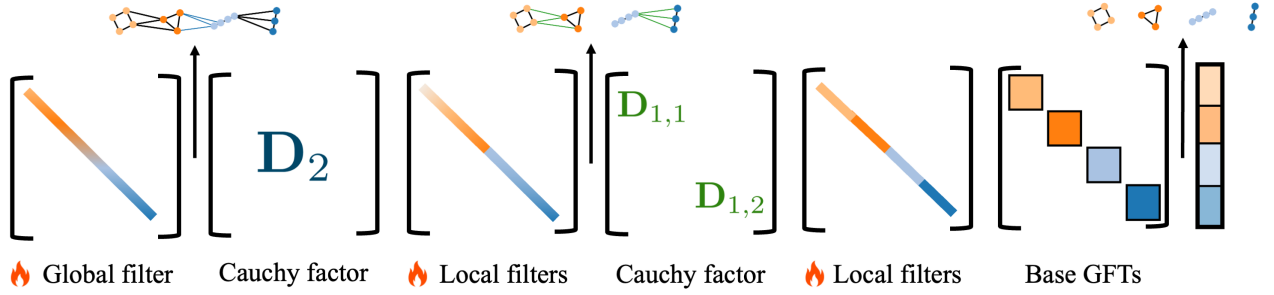


Figure 1. (a) A graph in $\mathcal{F}(L = 3, \{\mathcal{G}_i\}_{i=1}^8, k = 3)$. Bridge edges are shown in different colors. The GFT basis of the whole graph decomposes as the product of the GFT bases of the base graphs by a sequence of Cauchy factors. (b) We first compute the base GFTs of small subgraphs (right) and apply local spectral filters. We mix the outputs via Cauchy factors corresponding to bridge edges. After all merges, a global spectral filter acts on the full graph representation (left), allowing for global filtering while preserving locality.

GFT admits a hierarchical decomposition into the GFTs of the local *subgraphs* (a connected set of nodes) forming the partition, which are then multiplied by a series of *Cauchy matrices*, each arising from a rank-one update associated with a *cut* edge connecting two subgraphs (cf. Figure 1). Matrices in the Cauchy factorization are block-diagonal, hence localized to graph regions, and the composition of all terms yields the global GFT.

Computing this factorization is efficient: obtaining all terms has quadratic complexity in the number of nodes, scaled by the size of the subgraph cut. For graphs with hierarchical structure, common in real-world networks (Barthélemy, 2011; Clauset et al., 2008), this leads to significant computational savings over the cubic complexity eigendecomposition. While this complexity result holds for arbitrary graph partitions, the factorization cost may be high for graphs with large cuts. Hence, we introduce a graph decomposition algorithm that explicitly aims to minimize the cost of computing the Cauchy factorization. Specifically, we seek balanced partitions with small cuts, directly optimizing the theoretical factorization cost. When such favorable partitions are not available, we use spectral sparsification (Spielman and Srivastava, 2008) to reduce the cut size while preserving the spectral properties of the original graph. We validate this analysis through experiments on synthetic graphs, confirming the predicted runtime scaling and demonstrating substantial improvements over the dense eigendecomposition of the graph Laplacian.

This divide-and-conquer strategy is then used as the foundation for a new class of spectral GNNs, termed Local to Global Net (L2G-Net). As shown in Figure 1, at each stage, the outputs of each local GFT (i.e., subgraph spectral information) are processed using learnable filters before merging via Cauchy factors. In the last level, a global learnable spectral filter processes the entire graph signal without requiring a full eigendecomposition of the Laplacian (cf. Figure 1(b)). This allows L2G-Net to combine global spectral processing with locality biases by construction.

From a learning perspective, L2G-Net first analyzes the graph structure in a task-independent manner (by identifying subgraphs and sparsifying), and then builds an architecture tailored to the specific graph and learns its parameters. Thus, different graphs yield different hierarchical structures. This allows local filters to capture patterns within subgraphs, while the global filter models subgraph interactions. This property is relevant in long-range setups. MPNNs and polynomial approximations must relay information through bottlenecks, which leads to over-squashing (Alon and Yahav, 2021), while graph transformers (Dwivedi and Bresson, 2021) bypass this with learned attention but require additional inputs (e.g., positional encodings) to encode graph structure. L2G-Net offers a complementary inductive bias by encoding graph structure into the computation (Figure 2). We show a positioning table in Table 1; see Appendix B for detailed discussions.

We apply L2G-Net to 1) two transductive benchmarks

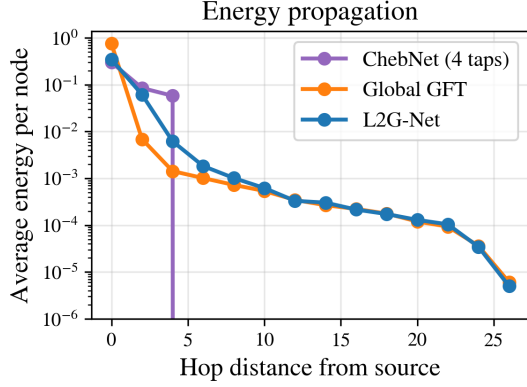


Figure 2. Average energy versus hop distance from the source when we input a unit impulse at each node of a graph. ChebNet (3rd-order) has a hard cutoff at 3 hops, while a GFT-based GNN propagates energy globally. L2G-Net’s locality bias adapts to the graph structure, interpolating between these two extremes.

Table 1. Comparison of different graph learning methods. CU stands for computational unit and GT for graph transformers.

Method	Cost	Locality	CU
MPNNs	$O(E)$	Local	Node
GT	$O(n^2)$	Global	Node
ChebNet	$O(K E)$	K -hop	Node
GFT	$O(n^3)$	Global	Graph
L2G-Net	$O(kn^2)$	Local to global	Subgraph

of large-scale graphs (Platonov et al., 2023; Liang et al., 2025), and 2) an inductive benchmark with smaller graphs (Dwivedi et al., 2022b). The datasets we consider emphasize non-local structural dependencies, where MPNNs are known to struggle due to their inductive biases (Alon and Yahav, 2021; Rusch et al., 2023; Hariri et al., 2026). Our results show that L2G-Net outperforms existing spectral strategies (Hariri et al., 2026), with lower complexity than applying the global GFT and a similar number of learned parameters. We achieve results competitive with state-of-the-art techniques using attention (Deng et al., 2024), while requiring orders of magnitude fewer learnable parameters.

Our contributions are: 1) showing that the GFT basis admits an exact Cauchy factorization for any partition, computable in quadratic time from the graph Laplacian without full eigendecompositions (Section 3); 2) an algorithm to find partitions that are computationally advantageous for Cauchy factorizations (Section 4); 3) L2G-Net, a spectral GNN family based on this factorization that combines local and global computations (Section 5).

2. Background

Notation. Uppercase and lowercase bold letters, such as $\mathbf{A}$ and $\mathbf{a}$, denote matrices and vectors, respectively. The n th

entry of $\mathbf{a}$ is a_n , and the (i, j) -th entry of $\mathbf{A}$ is A_{ij} .

2.1. Spectral graph theory

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{V})$ be a weighted undirected graph, with vertex set $\mathcal{V}$, edge set $\mathcal{E}$, weighted adjacency matrix $\mathbf{W}$, and self-loop diagonal matrix $\mathbf{V}$. The generalized graph Laplacian (GGL) is $\mathbf{L} \doteq \mathbf{D} - \mathbf{W} + \mathbf{V}$, where $\mathbf{D} \doteq \text{diag}(\mathbf{1}^\top \mathbf{W})$ is the degree matrix (Biyikoğlu et al., 2007). Given the eigendecomposition $\mathbf{L} = \mathbf{U} \text{diag}(\boldsymbol{\lambda}) \mathbf{U}^\top$, we refer to $\mathbf{U}^\top$ as the graph Fourier transform (GFT) basis (Chung, 1997; Ortega et al., 2018) associated with $\mathcal{G}$. Any graph GGL decomposes into a sum of rank-one terms (or “baby Laplacians”), one for each edge contribution.

Proposition 2.1 ((Batson et al., 2014)). *Let $\mathbf{e}_j$ be the j th canonical vector, for $j = 1, \dots, n$. Then, the GGL $\mathbf{L}$ of an undirected graph can be written as*

$$\mathbf{L} = \sum_{(i,j) \in \mathcal{E}} w_{ij} (\mathbf{e}_i - \mathbf{e}_j)(\mathbf{e}_i - \mathbf{e}_j)^\top + \sum_{(i,i) \in \mathcal{E}} v_{ii} \mathbf{e}_i \mathbf{e}_i^\top, \quad (1)$$

where w_{ij} denotes the edge weights, for $i, j = 1, \dots, |\mathcal{V}|$.

We use Proposition 2.1 to express the addition of an edge to a graph with Laplacian $\mathbf{L}$ as a *rank-one update*:

$$\tilde{\mathbf{L}} = \mathbf{L} + w_{ij} \mathbf{v} \mathbf{v}^\top, \quad (2)$$

with $\mathbf{v} = (\mathbf{e}_i - \mathbf{e}_j)$ when adding an edge between i and j .

2.2. Cauchy matrices

Definition 2.1 (Cauchy matrix (Gastinel, 1960)). Given two vectors $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^n$ with no common entries, the Cauchy matrix $\tilde{\mathbf{C}}(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{n \times n}$ has entries $\tilde{C}_{ij} = 1/(x_i - y_j)$ for $i, j = 1, \dots, n$.

Definition 2.2 (Orthogonal Cauchy-like matrix, OCLM (Fasino, 2023; Cai et al., 2018)). A matrix $\mathbf{C}(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{n \times n}$ is an orthogonal Cauchy-like matrix if it is orthogonal and can be written as $\mathbf{C}(\mathbf{x}, \mathbf{y}) = \text{diag}(\mathbf{s}) \tilde{\mathbf{C}}(\mathbf{x}, \mathbf{y}) \text{diag}(\mathbf{t})$ for some $\mathbf{s}, \mathbf{t} \in \mathbb{R}^n$.

OCLMs relate eigenvectors before and after a rank-one update of a symmetric matrix (Fasino, 2023). Each rank-one update corresponds to an edge in the GGL. In particular, for (2), with $\mathbf{L} = \mathbf{U} \text{diag}(\boldsymbol{\lambda}) \mathbf{U}^\top$ and $\tilde{\mathbf{L}} = \tilde{\mathbf{U}} \text{diag}(\tilde{\boldsymbol{\lambda}}) \tilde{\mathbf{U}}^\top$, it can be shown that (Fernández-Menduiña et al., 2025a):

$$\tilde{\mathbf{U}}^\top = -\mathbf{C}(\tilde{\boldsymbol{\lambda}}, \boldsymbol{\lambda}) \mathbf{U}^\top. \quad (3)$$

For rank-one updates, $\tilde{\boldsymbol{\lambda}}$ can be found via the secular equation (Golub, 1973) (cf. Appendix A). Relation (3) applies only to graphs without repeated eigenvalues and to updates that are not orthogonal to any column of $\mathbf{U}$ (cf. Appendix D).

2.3. Problem statement

We seek to construct spectral operators that compute the GFT of a given graph efficiently via localized computations. To this end, we rely on hierarchical decompositions into subgraphs connected by cuts of reduced size.

Definition 2.3 (Hierarchical graph family (HGF)). Let $\{\mathcal{G}_i\}_{i=1}^m$ be a collection of base graphs. A graph $\mathcal{G}$ belongs to the hierarchical graph family $\mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$ if it can be constructed by recursively merging pairs of subgraphs over L levels, where at each level at most k edges (bridge edges) are added between any pair of subgraphs.

Every graph belongs to at least one HGF: choose each base graph to be a singleton node and then add all edges between pairs of nodes. HGFs allow us to formalize the algorithmic complexity of the GFT computation: graphs exhibiting modular or multi-scale structure, which are typical in practice (Barthélemy, 2011), admit HGFs with small cut sizes, enabling efficient divide-and-conquer processing.

In this paper, given a graph $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$, we show that its GFT basis can be factorized into a sequence of Cauchy factors (Theorem 3.1), and we provide an $O(n^2k)$ algorithm to compute this factorization (Theorem 3.2). Moreover, for arbitrary graphs, we propose algorithms to identify hierarchies that optimize factorization complexity (Section 4). L2G-Net relies on this factorization, yielding a new class of spectral GNNs that avoids $O(n^3)$ eigendecompositions and introduces local inductive biases (Section 5).

3. Cauchy factorization of the GFT

We first generalize the result of (Fernández-Menduiña et al., 2025a) to remove the constraints on the eigenvalues and updates (see Appendix D for details) by introducing *Cauchy factors*, which will play a similar role to the Cauchy matrices in (3) for the case of arbitrary graphs. While we focus on the Laplacian case, the following results hold for any symmetric matrix.

Definition 3.1 (Cauchy factor (CF)). Let $\mathcal{S}$ denote the set of eigenvalue indices selecting 1) distinct eigenvalues, and among them, 2) those with corresponding eigenvectors not orthogonal to $\mathbf{v}$, and define the symmetric matrix $\tilde{\mathbf{L}} = \mathbf{L} + \rho \mathbf{v} \mathbf{v}^\top$. The associated Cauchy factor is

$$\mathbf{D}(\tilde{\boldsymbol{\lambda}}, \boldsymbol{\lambda}) = \mathbf{P}_\mathcal{S}^\top \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & -\mathbf{C}(\tilde{\boldsymbol{\lambda}}_\mathcal{S}, \boldsymbol{\lambda}_\mathcal{S}) \end{bmatrix} \mathbf{P}_\mathcal{S}, \quad (4)$$

where $\mathbf{C}(\tilde{\boldsymbol{\lambda}}_\mathcal{S}, \boldsymbol{\lambda}_\mathcal{S})$ is an OCLM, $\mathbf{I}$ is the identity on invariant spectral directions of dimension $(n - |\mathcal{S}|)$, and $\mathbf{P}_\mathcal{S}$ permutes the basis so that invariant and affected components are grouped together.

In this case, $\mathbf{C}(\tilde{\boldsymbol{\lambda}}_\mathcal{S}, \boldsymbol{\lambda}_\mathcal{S})$ corresponds to the OCLM associated with the deflated problem. The following result gener-

alizes (Fernández-Menduiña et al., 2025a) from path-graph Laplacians to arbitrary symmetric matrices.

Lemma 3.1 (Progressive decomposition identity). Let $\tilde{\mathbf{L}} = \mathbf{L} + \alpha \mathbf{v} \mathbf{v}^\top = \tilde{\mathbf{U}} \text{diag}(\tilde{\boldsymbol{\lambda}}) \tilde{\mathbf{U}}^\top$, with $\mathbf{L} = \mathbf{U} \text{diag}(\boldsymbol{\lambda}) \mathbf{U}^\top$. Then, generalizing (3), the updated eigenvectors are

$$\tilde{\mathbf{U}}^\top = \mathbf{D}(\tilde{\boldsymbol{\lambda}}, \boldsymbol{\lambda}) \mathbf{U}^\top. \quad (5)$$

Proof. See Theorem E.1 in the Appendix. $\square$

We can now extend this identity to HGFs.

Theorem 3.1 (Cauchy factorization). Let $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$ with Laplacian $\mathbf{L} = \mathbf{U} \text{diag}(\boldsymbol{\lambda}) \mathbf{U}^\top$. Let $\mathbf{U}_0$ denote the block-diagonal matrix, where each block contains the eigenvectors of one subgraph $\mathcal{G}_i$. Then,

$$\mathbf{U}^\top = \mathbf{D}(\boldsymbol{\lambda}, \tilde{\boldsymbol{\lambda}}_{K-1}) \cdots \mathbf{D}(\tilde{\boldsymbol{\lambda}}_1, \tilde{\boldsymbol{\lambda}}_0) \mathbf{U}_0^\top, \quad (6)$$

where $\tilde{\boldsymbol{\lambda}}_0$ are the eigenvalues of the partitioned graph, and $\tilde{\boldsymbol{\lambda}}_{K-\ell}$ follows by removing $\ell = 1, \dots, K$ bridge edges from $\mathcal{G}$, where $K = k(2^L - 1)$.

Proof. Sketch: By Proposition 2.1, the Laplacian of $\mathcal{G}$ can be written as a sequence of rank-one edge updates to connect the m subgraphs $\mathcal{G}_i$. Applying the progressive decomposition identity in Lemma 3.1 to each update and composing the resulting Cauchy factors yields the stated factorization. The full proof is given in Appendix E. $\square$

Intuitively, each Cauchy factor is a mixing step between two subgraphs joined by an edge. An edge addition is a rank-one update of the Laplacian, which rotates the eigenbasis by a Cauchy matrix; since the update only affects the two endpoints' subgraphs, the rotation mixes their spectral coefficients and leaves the rest unchanged. The factorization in Theorem 3.1 builds the global GFT by chaining these steps: start from independent subgraph GFTs, and merge one cut at a time. We show now that this factorization can be computed in quadratic time in the number of nodes, scaled by the cut size. Let $T_m(\cdot)$ be the time complexity of a parallel algorithm, assuming m processors are available (Jájá, 1992). Then, we have the following result.

Theorem 3.2. Let $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$. Assume the eigendecomposition of leaf subgraph $\mathcal{G}_i$ can be computed in $O(f_i(n))$ time. Then, we can find the Cauchy decomposition of the eigenvectors basis of the graph Laplacian $\mathbf{L}$ in $O(kn^2 + \sum_i f_i(n))$ for a sequential solver and in time $T_m(n, k) = O(kn^2 + \max_i f_i(n))$ for a parallel solver.

Proof. See Appendix F. $\square$

For simplicity, we focus on the parallel case. We highlight two cases: 1) when $L = \log_2 n$, the base graphs become

Algorithm 1 Cauchy factorization

Input: HGF graph $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$.

Output: Eigenvalues λ , Cauchy history $\mathcal{H}$.

```

1: 1. Initialization
2: Compute leaf eigendecompositions  $\{(\mathbf{U}_i, \lambda_i)\}$ .
3: Initialize set of transformed updates,  $\mathcal{Z} = \emptyset$ .
4: for each bridge edge  $e = (u, v)$  do
5:    $\mathcal{Z} \leftarrow \mathcal{Z} \cup (\mathbf{e}_u - \mathbf{e}_v)$ 
6: Project all  $\mathbf{z} \in \mathcal{Z}$  into the leaf bases  $\mathbf{U}_i$ .

7: 2. Hierarchical merge
8: for level  $\ell = 1$  to  $L$  do
9:   for each pair of subgraphs to merge do
10:    Concatenate eigenvalues:  $\lambda \leftarrow [\lambda_L; \lambda_R]$ .
11:    for each bridge edge  $e$  connecting them do
12:      Retrieve projection vector  $\mathbf{z} \in \mathcal{Z}$ .
13:      Solve the secular equation to get  $\lambda_{\text{new}}$ .
14:      Construct OCLM  $\mathbf{C}$  from  $(\mathbf{z}, \lambda, \lambda_{\text{new}})$ .
15:      Update relevant  $\mathbf{z}' \in \mathcal{Z}$  via  $\mathbf{z}' \leftarrow \mathbf{C}^\top \mathbf{z}'$ .
16:       $\lambda \leftarrow \lambda_{\text{new}}, \quad \mathcal{H} \leftarrow \mathcal{H} \cup \{\mathbf{C}\}$ 
17: return  $\lambda, \mathcal{H}$ 
    
```

trivial and the complexity is $O(kn^2)$, and 2) when $k \ll n, k = O(1)$, i.e., connections between graphs are sparse, the complexity becomes $O(n^2 + \max_i f_i(n))$. An outline of the algorithm is depicted in Algorithm 1, and the full algorithm is shown in Algorithm 3. Next, we show how to find a suitable partition for a given graph.

4. Finding a suitable HGF

The complexity of our method scales with the cut size k , not with the edge count; sparse graphs help insofar as they admit small-cut partitions. Although every graph admits an HGF decomposition, when $k = O(n)$ the decompositions may not be favorable for computation. We provide a greedy algorithm that partitions any graph into subgraphs directly minimizing the cost of computing the Cauchy factorization. When decomposition yields diminishing returns, we use spectral sparsification to reduce the cut size.

4.1. Greedy HGF construction

Based on the GFT factorization cost (Theorem 3.2), we develop a heuristic that favors balanced partitions with few bridge edges. For a given graph, we generate candidate partitions using balanced cuts with varying balance constraints (e.g., spectral bisection based on the Fiedler vector with varying balance tolerances) (Ng et al., 2001). To accept a partition, we evaluate whether the split offers computational benefits based on Theorem 3.2. Let $f(\mathcal{G})$ denote the theoretical complexity cost of processing graph $\mathcal{G}$, i.e.,

the eigendecomposition complexity. Given a graph $\mathcal{G}$ and a candidate split into subgraphs $\mathcal{G}_1, \mathcal{G}_2$, we accept the partition only if:

$$n^2 k + \max_{i=1,2} f(\mathcal{G}_i) < f(\mathcal{G}). \quad (7)$$

If this condition holds, we recur on $\mathcal{G}_1$ and $\mathcal{G}_2$. If no balanced cut satisfies this condition, we mark the subgraph as a leaf or apply the sparsification strategy below.

Regarding spectral bisection, using the Lanczos algorithm, each iteration requires a matrix-vector multiplication with the Laplacian, which costs $O(|E|)$ time for sparse graphs. Since we have to repeat this operation on each level, the total runtime is $O(|E| 2^L)$, with $L = \log(n)$. Hence, computing the Fiedler vector is nearly linear in the number of edges and is not a computational bottleneck.

4.2. Cut sparsification

When partitioning fails to reduce the cost function, rather than keeping all edges crossing the cut, we remove some of them via a spectral sparsification (Spielman and Srivastava, 2008) applied locally, i.e., only to subgraph cuts.

Theorem 4.1 (Interface sparsification for HGFs). *Let $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$ and $\varepsilon \in (0, 1)$. Then there exists a graph $\mathcal{G}' \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k')$ such that their combinatorial Laplacians $\mathbf{L}$ and $\mathbf{L}'$ satisfy $\forall \mathbf{x} \in \mathbb{R}^n$,*

$$(1 - \varepsilon) \mathbf{x}^\top \mathbf{L} \mathbf{x} \leq \mathbf{x}^\top \mathbf{L}' \mathbf{x} \leq (1 + \varepsilon) \mathbf{x}^\top \mathbf{L} \mathbf{x}, \quad (8)$$

with $k' \leq O(\varepsilon^{-2})$ that can be constructed in $O(|E| \log n)$.

Proof. Fix the hierarchical partition defining $\mathcal{G}$. At each level, consider the subgraph induced by the bridge edges connecting pairs of subgraphs. Applying spectral sparsification (Spielman and Srivastava, 2008) independently to each such interface yields a graph with at most $k' = O(\varepsilon^{-2})$ edges per interface, while preserving the Laplacian quadratic form within $(1 \pm \varepsilon)$. Since sparsification preserves vertex partition, the resulting graph $\mathcal{G}'$ belongs to the same HGF with reduced interface size. $\square$

By Theorem 4.1, any graph can be replaced by a spectrally equivalent member of an HGF with bounded cut size, ensuring that the complexity bounds of Theorem 3.2 apply up to a controllable approximation error. From the perspective of Cauchy factorization, sparsification can be viewed as a theoretically grounded form of truncation, limiting the effective number of the Cauchy updates while preserving the Laplacian quadratic form up to a controlled error. For spectral filters $g(\cdot)$ that are Lipschitz continuous on the spectrum of $\mathbf{L}$, the output of a layer based on $\mathbf{L}'$ deviates from that based on $\mathbf{L}$ by at most $O(\varepsilon)$, with the constant depending on the Lipschitz constant of $g(\cdot)$ (Levie et al., 2021).

5. L2G-Net

Given the Cauchy factorization, we can add learnable spectral filters in between Cauchy factors while preserving the overall complexity and providing more flexibility than GFT-based spectral GNNs. This is the idea underpinning L2G-Net (Figure 1b). L2G-Net can be used as a drop-in replacement for standard spectral GNNs, potentially involving pointwise non-linearities and residual connections.

Assume a stack of M spectral layers, acting over $\mathbf{X}_0 \in \mathbb{R}^{n \times n_f}$, where n_f is the number of channels. Let $\mathbf{Z}_m = \mathbf{X}_m \mathbf{W}_m$. Then, for $c = 1, \dots, n_f$,

$$\mathbf{X}_{m+1,c} = \mathbf{U} g_{\theta,c}(\lambda) \mathbf{U}^\top \mathbf{Z}_{m,c}, \quad m = 0, \dots, M-1. \quad (9)$$

At a high level, our construction modifies the forward transform $\mathbf{U}^\top \mathbf{Z}_{m,c}$ by applying spectral processing independently on small base subgraphs using learnable local graph filters. Then, information is progressively mixed across larger graph components through a sequence of Cauchy factors (cf. Theorem 3.1). Finally, the synthesis transform is computed, again using the same factorization.

We start from the Cauchy factorization in Theorem 3.1. We insert learnable local filters before and after multiplying by the set of Cauchy factors corresponding to the cuts between each pair of subgraphs. Let these filters be $g_{r,p}(\cdot)$ for the r th level, with $r = 0, \dots, L$, and considering the p th pair of subgraphs, with $p = 1, \dots, 2^r - 1$. Let $\mathbf{D}_{r,p}$ be the product of the Cauchy factors corresponding to the same cut. Given the filtered spectral representations at level $r-1$ on each subgraph, $\mathbf{H}_{p,1}^{r-1}$ and $\mathbf{H}_{p,2}^{r-1}$, we compute:

$$\mathbf{H}_p^{(r)} = g_{r,p}(\lambda_{r,p}) \mathbf{D}_{r,p} \begin{bmatrix} \mathbf{H}_{p_1}^{(r-1)} \\ \mathbf{H}_{p_2}^{(r-1)} \end{bmatrix}, \quad (10)$$

where $\lambda_{r,p}$ are the eigenvalues of the merged subgraph at level r . By repeating this procedure recursively up to $r = L$, we obtain the spectral representation $\mathbf{H}^{(L)}$. We will denote by $\mathbf{U}(\Phi)$ the matrix replacing $\mathbf{U}$ with the concatenation of GFTs, filtering, and Cauchy matrices, where Φ represents learnable parameters for all r and p . The output in the node domain is given by $\mathbf{X}_{\text{out}} = \mathbf{U} g_{\theta}(\lambda) \mathbf{U}(\Phi) \mathbf{X}_m$. We remark that products by $\mathbf{U}$ are computed using the factorization, so $\mathbf{U}$ is never explicitly computed.

Remarkably, the partition and Cauchy factorization depend only on $\mathcal{G}$; learnable parameters appear only in the filters $g_{r,p}(\cdot)$, so the architecture is fixed by the graph before any training begins. The following result characterizes the expressiveness of L2G-Net.

Theorem 5.1 (Expressiveness). *Let $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$ with $k \geq 1$. The class of operators realizable by L2G-Net strictly contains the class of global spectral filters $g(\mathbf{L})$.*

Table 2. Average test accuracy with standard deviation for 10 random train/test partitions on the benchmark in (Platonov et al., 2023). We report the best-performing methods per family. We show average Acc for *Roman-empire* (R. Emp.) and *Amazon-ratings* (Am. Rat.) and AUC for *Tolokers* (Tolok.) and *Minesweeper* (Mines.). Higher is better ($\uparrow$). See also Table 12.

Method	R. Emp.	Am. Rat.	Mines.	Tolok.
GCN	73.69(0.7)	48.70(0.6)	89.75(0.5)	83.64(0.7)
CO-GNN	91.57(0.3)	54.17(0.4)	97.31(0.4)	84.45(1.2)
Polynormer	92.55 (0.3)	54.81 (0.5)	97.46(0.4)	85.91 (0.7)
MP-SSM	90.91(0.5)	53.65(0.7)	95.33(0.7)	85.26(0.9)
St-ChebNet	92.03(0.9)	53.15(0.2)	95.71(2.3)	85.55(3.4)
Ours	<u>92.12</u> (1.1)	53.39(0.6)	97.50 (0.3)	<u>85.57</u> (0.6)

Table 3. Runtime (s) for computing the eigenvalue decomposition (ED) and the Cauchy factorization (CF) on heterophilous graphs from (Platonov et al., 2023).

Method	R. Emp.	Am. Rat.	Mines.	Tolok.
ED	731.12	896.56	92.34	117.57
CF (ours)	232.49	281.30	32.21	91.33

Proof. See Appendix G. $\square$

A spectral filter $g(\mathbf{L})$ applies the same response across the entire graph. L2G-Net applies different spectral responses to different subgraphs before merging via Cauchy factors, producing operators that cannot be represented as $g(\mathbf{L})$. This degree of freedom is given by the partition structure.

6. Experiments

First, we validate our theoretical complexity predictions on synthetic graphs with controlled structure. Second, we evaluate our method on real-world graphs that may deviate from the ideal hierarchical setup. While our primary contribution is to provide a principled framework for exploiting hierarchical graph structure when it is present, the GNN experiments demonstrate that the approach degrades gracefully under approximation. All experiments are conducted on an Intel(R) Core(TM) i9-9900K CPU @ 3.60GHz with an NVIDIA GeForce RTX 2070 GPU.

6.1. Synthetic experiments

We validate the predictions of Theorem 3.2 measuring the runtime of the proposed Cauchy factorization on synthetic graphs. We isolate the dependence on graph size and cut size. All experiments are run on CPU.

Setup. We generate random Barabási-Albert graphs (Barabási and Albert, 1999). For each graph with n nodes, we first compute a balanced partition using spectral bisection based on the Fiedler vector of the combinatorial Lapla-

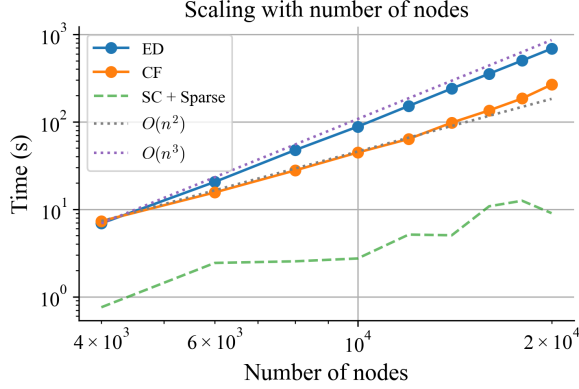


Figure 3. Runtime on a random graph for eigendecomposition (ED), Cauchy factorization (CF), and preprocessing (SC+Sparse). The Cauchy factorization scales quadratically with the number of nodes. The preprocessing time is negligible.

cian. We repeat this on each subgraph to obtain a partition with 4 subgraphs. To control the cut size, we sparsify the cut edges, retaining a fixed target number of bridge edges.

Runtime vs graph size (Figure 3). We compare (i) full eigendecomposition of the Laplacian via `numpy.linalg.eigh` and (ii) our Cauchy factorization. We report the median runtime across three runs vs number of nodes n for a fixed cut size $k = 5$. Eigendecomposition scales with $O(n^3)$ and becomes impractical beyond $n \approx 10^4$. The Cauchy factorization follows a clear quadratic trend, consistent with the $O(kn^2)$ complexity predicted by our analysis. As n increases, the cost of the base eigendecompositions of the individual subgraphs becomes non-negligible, causing the overall trend to gradually approach cubic behavior. This effect is expected and can be mitigated by increasing the number of partitions; when the number of levels is $L = \log n$, the complexity of the base eigendecompositions is trivial. The preprocessing overhead (spectral cut and sparsification) remains negligible across all graph sizes considered.

Runtime vs cut size (Figure 4). For a fixed graph size ($n = 8000$), the runtime of the proposed Cauchy factorization grows linearly with the number of crossing edges, k , which corresponds to the number of the Cauchy updates. This confirms that the computational cost is governed by the cut size rather than by the total graph size. The preprocessing step based on spectral cuts and sparsification exhibits an approximately constant runtime, as promised by Theorem 4.1. These results show that our method remains efficient because you can sparsify first to reduce the cut size and this sparsification has limited complexity.

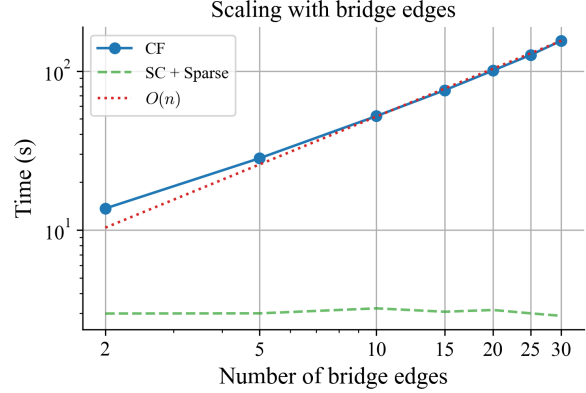


Figure 4. Runtime against k on a random graph for Cauchy factorization (CF) and preprocessing (SC+Sparse). The complexity of the Cauchy factorization scales linearly with the cut size. The preprocessing time is small and roughly constant with k .

6.2. Transductive heterophilous benchmarks

Setup. We evaluate the proposed method on a heterophilous benchmark suite (Platonov et al., 2023), which comprises large-scale graphs designed to challenge MPNNs. In particular, we focus on *Roman-empire*, *Amazon-ratings*, *Minesweeper*, and *Tolokers*. These datasets combine non-local dependencies with graph sizes for which the global GFT remains feasible but is very slow, making them well-suited for assessing our approach. We rely on the normalized Laplacian, although we sparsify based on the combinatorial Laplacian (Batson et al., 2014). We use splines to parametrize our filters. Our method first partitions the graph into two subgraphs (one level of hierarchy) and sparsifies the cut. A single shared filter is used across all feature channels and layers (cf. Appendix C).

Performance (Table 2). L2G-Net achieves competitive or superior performance across all benchmarks, exceeding the best baselines (Deng et al., 2024) on *Minesweeper* and remaining close to the strongest methods on the others. Notably, while attention-based models such as Polynormer attain strong accuracy, they do so at the cost of learned graph structures that are harder to interpret (cf. Figure 6).

Runtime (Table 3). We report the cost of CF and the eigendecomposition. CF yields speedups across all datasets. *Tolokers* is a worst-case scenario: this graph is particularly dense, so the number of edges between subgraphs remains high after sparsification. Even in this case, we obtain runtime gains over direct eigendecomposition.

Learnable parameters (Figure 5). We compare Global GFT, L2G-Net, and Polynormer across the four datasets. The topological inductive bias allows L2G-Net to compete with state-of-the-art methods while using orders of magni-

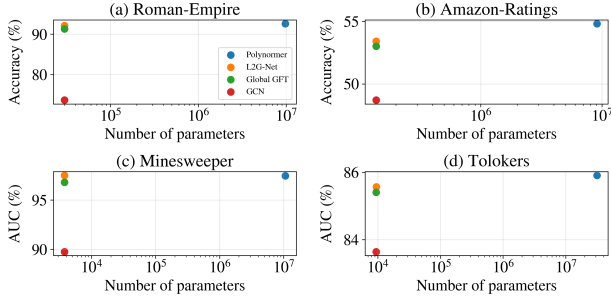


Figure 5. Performance vs complexity comparison. L2G-Net achieves competitive or superior performance with orders of magnitude fewer learnable parameters than state-of-the-art baselines, and outperforms the Global GFT across all datasets.

Table 4. Time (minutes) to compute the eigendecomposition (ED) versus the CF factorization on City-Networks. ED runs out of memory in all cases; EDP extrapolates the $O(n^3)$ trend.

Method	Paris	Shanghai	LA	London
ED	OOM	OOM	OOM	OOM
EDP	50412.82	121932.43	131543.54	1632123.32
CF	17.91	45.61	61.53	144.22

tude fewer learnable parameters. Moreover, L2G-Net consistently outperforms the Global GFT baseline across all datasets, highlighting the benefit of localized spectral processing over fully global spectral representations.

6.3. Long-range inductive benchmarks

To demonstrate the effectiveness of our method on inductive tasks, we evaluate L2G-Net on the long-range molecular benchmarks *peptides-func* and *peptides-struct* from (Dwivedi et al., 2022b), which consist of inductive graph-level prediction tasks on small molecular graphs (Table 6). These datasets allow for full Laplacian eigendecomposition; hence, we can isolate architectural inductive bias without confounding results with sparsification effects. Results show that L2G-Net performs within one standard deviation of state-of-the-art long-range models, including spatio-spectral hybrids such as (Geisler et al., 2024), without relying on positional encodings or graph rewiring. Moreover, L2G-Net outperforms spectral methods such as Specformer, ChebNet, Stable ChebNet, and ChebNetII. This indicates that the hierarchical spectral factorization underlying L2G-Net captures long-range dependencies in inductive settings without additional structural augmentation. Furthermore, our method can also be extended with positional encodings (PEs): when considering Laplacian and Random Walk PEs (Dwivedi et al., 2022a), we can obtain better accuracy in both benchmarks, demonstrating that L2G-Net and PEs are complementary.

6.4. Long-range transductive benchmarks

We consider the dataset from (Liang et al., 2025), which contains graphs with more than 10^5 nodes. We use different levels of decomposition ($L = 4$ for Paris and Shanghai, $L = 5$ for LA, and $L = 6$ for London). We sparsify keeping only 2 edges between each pair of subgraphs. Table 4 reports the time required to compute the factorization. Full eigendecomposition (ED) is infeasible on these graphs due to out-of-memory (OOM) errors; we include a projected ED runtime obtained by extrapolating the cubic scaling trend from smaller graphs, to give a sense of the asymptotic gap. The Cauchy factorization is three-four orders of magnitude faster than projected ED. The largest graph in the suite (London, 569k nodes) is factorized in roughly 144 minutes, which makes exact spectral processing tractable at scales where it was previously out of reach.

We also show accuracy results (Table 7). L2G-Net is competitive with state-of-the-art methods, and outperforms spectral baselines such as ChebNet. These results show that 1) L2G-Net scales to graphs where computing the GFT basis is unfeasible, and 2) the local-to-global inductive bias yields gains in long-range setups.

6.5. Local processing

Setup. To analyze the localization of predictive importance across graph nodes, we perform a node-level attribution study using Grad-CAM (Selvaraju et al., 2017) on the *Minesweeper* dataset. For each evaluation sample, we compute Grad-CAM scores at the node level, normalize them to sum to one, and sort nodes by decreasing importance. We then compute the cumulative sum of importance as a function of the fraction of nodes retained, yielding a cumulative contribution curve per sample. These curves are finally averaged across the evaluation set. We include results for other graphs in Appendix I.2.

Results (Figure 6). L2G-Net concentrates its predictive contributions on a small subset of nodes, reflecting its local bias. The Global GFT baseline spreads importance over a larger portion of the graph, consistent with global spectral representations. For Polynormer (Deng et al., 2024), graph learning combined with attention-based mechanisms leads to more diffuse contributions, making explanations difficult to interpret with respect to the input geometry. We also show the filters learned by L2G-Net in Appendix I.1.

6.6. Memory consumption

Storing a Cauchy matrix has complexity $O(n)$. Table 5 compares the memory required by the full GFT against our Cauchy factorization. On (Platonov et al., 2023), where

Table 5. Memory consumption (GB) of full GFT vs. our Cauchy factorization (CF), using 64-bit precision. On (Platonov et al., 2023) we use $L = 1$; on City-Networks (Liang et al., 2025), where full GFT is infeasible at practical hardware limits, we use deeper hierarchies.

Method	Mines.	Tolok.	Am. Rat.	R. Emp.	Paris	Shanghai	LA	London
Full GFT	0.80	1.11	4.80	4.11	103.97	270.61	463.06	2588.22
CF (ours)	0.20	0.28	1.21	1.03	12.21	16.98	14.58	40.96

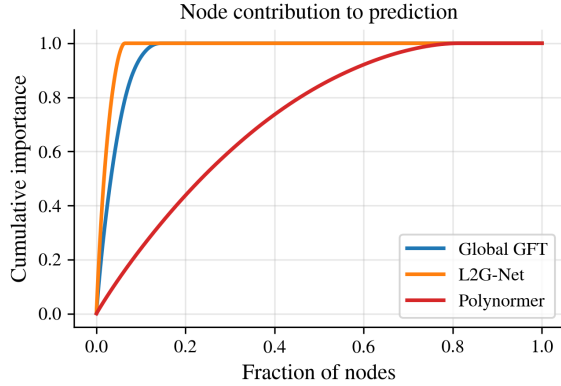


Figure 6. Cumulative node contribution to the prediction on Minesweeper. Average across validation set; shaded areas indicate confidence intervals. L2G-Net localizes predictive power on a smaller fraction of nodes than Global GFT. The node contributions of Polynormer are less localized in the original graph.

both methods are tractable, the Cauchy factorization reduces memory by roughly $4\times$ across all four datasets at the shallowest setting ($L = 1$). The gap becomes more clear on City-Networks: storing the full GFT is well beyond standard hardware. Our factorization brings these costs down substantially, making exact spectral processing feasible.

7. Conclusion

We introduced an exact factorization of the GFT for arbitrary graphs based on Cauchy matrices and graph partitions. This factorization enables a divide-and-conquer strategy for spectral processing, reducing the cost of exact GFT-based operators from $O(n^3)$ to $O(n^2)$, scaled by the subgraph cut size. We proposed an algorithm to find partitions that minimize the factorization cost and, when this remains costly, reduce complexity via cut sparsification. Building on these results, we introduced L2G-Net, a new class of spectral GNNs that combine local and global spectral processing by construction. L2G-Net is most appropriate in regimes where spectral processing is desired (e.g., long-range dependencies) and polynomial filters would require high order or deep architectures, leading to oversquashing or numerical instability. Experiments on synthetic and real-world benchmarks validate the theoretical analysis, confirming the predicted runtime scaling and showing that the resulting models remain competitive while requiring fewer parameters than transformer-based meth-

Table 6. LRGB results (Dwivedi et al., 2022b); top performers per category shown, full results in Appendix I. AP on peptides-func (higher is better), MAE on peptides-struct (lower is better). Trans. = transformer, Rewir. = rewiring, SS = state space.

Type	Model	pep-func ($\uparrow$)	pep-struct ($\downarrow$)
Trans.	Graph ViT	69.42 ± 0.75	0.2449 ± 0.0016
	GRIT	69.88 ± 0.82	0.2460 ± 0.0012
Rewir.	DRew-GCN+PE	71.50 ± 0.44	0.2536 ± 0.0015
SS	GMN	70.71 ± 0.83	0.2473 ± 0.0025
	MP-SSM	69.93 ± 0.52	0.2458 ± 0.0017
GNN	GCN	68.60 ± 0.50	0.2460 ± 0.0007
	GRAMA	70.93 ± 0.78	0.2436 ± 0.0022
	PH-DGN	70.12 ± 0.45	0.2465 ± 0.0020
	S2GCN	72.75 ± 0.66	0.2467 ± 0.0019
	+PE	73.11 ± 0.66	0.2447 ± 0.0032
Ours	L2G-Net	72.14 ± 0.24	0.2479 ± 0.0012
	+PE	72.46 ± 0.25	0.2462 ± 0.0011

Table 7. Average test accuracy with standard deviation for 4 random seeds on City-Networks (Liang et al., 2025). Higher is better. We show selected methods, full results in Appendix I.

Model	Paris	Shanghai	LA	London
ChebNet	$54.1_{(0.2)}$	$66.5_{(0.1)}$	$61.4_{(0.4)}$	$54.7_{(0.2)}$
GCN	$53.2_{(0.3)}$	$62.1_{(0.2)}$	$58.3_{(0.3)}$	$50.1_{(0.7)}$
SAGE	$54.6_{(0.2)}$	$68.3_{(0.5)}$	$61.4_{(0.3)}$	$55.4_{(0.2)}$
GraphGPS	$52.1_{(0.6)}$	$63.0_{(0.5)}$	$59.8_{(0.5)}$	OOM
Expformer	$55.1_{(0.8)}$	$70.2_{(0.4)}$	$63.8_{(0.6)}$	$49.5_{(0.4)}$
SGFormer	$52.0_{(0.8)}$	$64.1_{(0.3)}$	$60.1_{(0.7)}$	$48.3_{(0.3)}$
Ours	$55.4_{(0.4)}$	$69.8_{(0.5)}$	$63.5_{(0.7)}$	$55.6_{(0.6)}$

ods and lower preprocessing cost than the global GFT. In strictly large-scale settings where linear complexity is key, polynomial filters and message passing remain preferable.

The factorization extends beyond the static undirected setting. For dynamic undirected graphs, edge insertions and deletions can be handled by adding a single Cauchy factor, while node additions require recomputing only the affected subgraph and its cut factors. Signed graphs are also supported since our result holds for any symmetric matrix; directed graphs (e.g., via Hermitian Laplacians) are left for future work. L2G-Net opens new directions for fast and structured spectral algorithms on graphs.

Acknowledgements

The authors would like to thank Jhony Giraldo for feedback on the initial versions of this manuscript. SFM would like to thank Narjes Nourzad for help designing Figure 1.

Impact Statement

This paper presents work aimed at advancing the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations*, 2021.
- Álvaro Arroyo, Alessio Gravina, Benjamin Gutteridge, Federico Barbero, Claudio Gallicchio, Xiaowen Dong, Michael Bronstein, and Pierre Vandergheynst. On vanishing gradients, over-smoothing, and over-squashing in GNNs: Bridging recurrent and graph learning. *Advances in Neural Information Processing Systems*, 38:74356–74393, 2026.
- Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999.
- Federico Barbero, Ameya Velingker, Amin Saberi, Michael M Bronstein, and Francesco Di Giovanni. Locality-aware graph rewiring in GNNs. In *International Conference on Learning Representations*, 2023.
- Marc Barthélemy. Spatial networks. *Physics reports*, 499(1-3):1–101, 2011.
- Joshua Batson, Daniel A Spielman, and Nikhil Srivastava. Twice-Ramanujan sparsifiers. *SIAM Rev.*, 56(2):315–334, 2014.
- Türker Biyikoğlu, Josef Leydold, and Peter F Stadler. *Laplacian eigenvectors of graphs: Perron-Frobenius and Faber-Krahn type theorems*. Springer, 2007.
- Michael M. Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. Geometric deep learning: Going beyond Euclidean data. *IEEE Signal Process. Mag.*, 34(4):18–42, 2017. doi: 10.1109/MSP.2017.2693418.
- Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann Lecun. Spectral networks and locally connected networks on graphs. In *International Conference on Learning Representations*, 2014.
- James R. Bunch, Christopher P. Nielsen, and Danny C. Sorensen. Rank-one modification of the symmetric eigenproblem. *Numerische Mathematik*, 31:31–48, 1978.
- Difeng Cai, Edmond Chow, Lucas Erlandson, Yousef Saad, and Yuanzhe Xi. SMASH: Structured matrix approximation by separation and hierarchy. *Numerical Linear Algebra with Applications*, 25(6), 2018.
- Ben Chamberlain, James Rowbottom, Maria I Gorinova, Michael Bronstein, Stefan Webb, and Emanuele Rossi. GRAND: Graph neural diffusion. In *International Conference on Machine Learning*, pages 1407–1418. PMLR, 2021.
- Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In *International Conference on Machine Learning*, 2020.
- Fan RK Chung. *Spectral graph theory*, volume 92. American Mathematical Soc., 1997.
- Aaron Clauset, Cristopher Moore, and Mark EJ Newman. Hierarchical structure and the prediction of missing links in networks. *Nature*, 453(7191):98–101, 2008.
- J. J. Cuppen. A divide and conquer method for the symmetric tridiagonal eigenproblem. *Numer. Math.*, 36(2):177–195, June 1980.
- Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in neural information processing systems*, 29, 2016.
- Chenhui Deng, Zichao Yue, and Zhiru Zhang. Polynormer: Polynomial-expressive graph transformer in linear time. In *International Conference on Learning Representations*, 2024.
- Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. *AAAI Workshop on Deep Learning on Graphs: Methods and Applications*, 2021.
- Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Graph neural networks with learnable structural and positional representations. In *International Conference on Learning Representations (ICLR)*, 2022a.
- Vijay Prakash Dwivedi, Ladislav Rampásek, Michael Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. Long range graph benchmark. *Advances in Neural Information Processing Systems*, 35:22326–22340, 2022b.

- Dario Fasino. Orthogonal Cauchy-like matrices. *Numerical Algorithms*, 92(1):619–637, 2023.
- Samuel Fernández-Menduiña, Eduardo Pavez, and Antonio Ortega. Fast DCT+: A family of fast transforms based on rank-one updates of the path graph. In *Proc. IEEE Int. Conf. Acoust., Speech, and Signal Process.* IEEE, 2025a.
- Samuel Fernández-Menduiña, Eduardo Pavez, Antonio Ortega, Tsung-Wei Huang, Thuong Nguyen Canh, Guan-Ming Su, and Peng Yin. Int-dtt+: Low-complexity data-dependent transforms for video coding. *arXiv preprint arXiv:2511.17867*, 2025b.
- Thomas Frerix and Joan Bruna. Approximating orthogonal matrices with effective Givens factorization. In *Intl. Conf. on Mach. Learn.*, pages 1993–2001. PMLR, 2019.
- Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv preprint arXiv:1810.05997*, 2018.
- N Gastinel. Inversion d’une matrice generalisant la matrice de Hilbert. *Chiffres*, 3:149–152, 1960.
- Simon Markus Geisler, Arthur Kosmala, Daniel Herbst, and Stephan Günnemann. Spatio-spectral graph neural networks. *Advances in Neural Information Processing Systems*, 37:49022–49080, 2024.
- Jhony H Giraldo, Konstantinos Skianis, Thierry Bouwmans, and Fragkiskos D Malliaros. On the trade-off between over-smoothing and over-squashing in deep graph neural networks. In *Proceedings of ACM international conference on information and knowledge management*, pages 566–576, 2023.
- Gene H Golub. Some modified matrix eigenvalue problems. *SIAM Rev.*, 15(2):318–334, 1973.
- Ming Gu and Stanley C. Eisenstat. A divide-and-conquer algorithm for the symmetric tridiagonal eigenproblem. *SIAM Journal on Matrix Analysis and Applications*, 16(1):172–191, 1995.
- Ming Gu and Stanley C. Eisenstat. Efficient algorithms for computing a strong rank-revealing QR factorization. *SIAM Journal on Scientific Computing*, 17(4):848–869, 1996.
- Ali Hariri, Alvaro Arroyo, Alessio Gravina, Moshe Eliasof, Carola-Bibiane Schönlieb, Davide Bacciu, Xiaowen Dong, Kamyar Azizzadenesheli, and Pierre Vandergheynst. Return of ChebNet: Understanding and improving an overlooked GNN on long range tasks. *Advances in Neural Information Processing Systems*, 38:136166–136196, 2026.
- Mingguo He, Zhewei Wei, Hongteng Xu, et al. Bernnet: Learning arbitrary graph spectral filters via Bernstein approximation. *Advances in neural information processing systems*, 34:14239–14251, 2021.
- Yinan Huang, William Lu, Joshua Robinson, Yu Yang, Muhan Zhang, Stefanie Jegelka, and Pan Li. On the stability of expressive positional encodings for graphs. In *International Conference on Learning Representations*, pages 39745–39774, 2024.
- Laurent Jacques. A quantized Johnson–Lindenstrauss lemma: The finding of Buffon’s needle. *IEEE Transactions on Information Theory*, 61(9):5012–5027, 2015.
- Joseph JáJá. *Parallel algorithms*. 1992.
- Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- Luc Le Magoarou, Rémi Gribonval, and Nicolas Tremblay. Approximate fast graph Fourier transforms via multi-layer sparse approximations. *IEEE Trans on Sign. and Inform. Process. over Netw.*, 4(2):407–420, 2018.
- Ron Levie, Federico Monti, Xavier Bresson, and Michael M Bronstein. Cayleynets: Graph convolutional neural networks with complex rational spectral filters. *IEEE Transactions on Signal Processing*, 67(1):97–109, 2018.
- Ron Levie, Wei Huang, Lorenzo Bucci, Michael Bronstein, and Gitta Kutyniok. Transferability of spectral graph convolutional neural networks. *Journal of Machine Learning Research*, 22(272):1–59, 2021.
- Guohao Li, Matthias Müller, Ali Thabet, and Bernard Ghanem. DeepGCNs: Can GCNs go as deep as CNNs? In *IEEE/CVF International Conference on Computer Vision*, pages 9266–9275, 2019.
- Ren-Cang Li. Solving secular equations stably and efficiently. Technical Report UCB/CSD-94-851, Dec 1994. URL <http://www2.eecs.berkeley.edu/Pubs/TechRpts/1994/5882.html>.
- Huidong Liang, Haitz Sáez de Ocáriz Borde, Baskaran Sripathmanathan, Michael Bronstein, and Xiaowen Dong. Towards quantifying long-range interactions in graph machine learning: a large graph dataset and a measurement. *arXiv preprint arXiv:2503.09008*, 2025.
- Andrew Ng, Michael Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. In T. Dietterich, S. Becker, and Z. Ghahramani, editors, *Proc. Advan. Neural Inf. Process. Syst.*, volume 14. MIT Press, 2001.

- Antonio Ortega, Pascal Frossard, Jelena Kovačević, José MF Moura, and Pierre Vandergheynst. Graph signal processing: Overview, challenges, and applications. *Proc. IEEE*, 106(5):808–828, 2018.
- Xiaofeng Ou and Jianlin Xia. SuperDC: Superfast divide-and-conquer eigenvalue decomposition with improved stability for rank-structured matrices. *SIAM Journal on Scientific Computing*, 44(5):A3041–A3066, 2022.
- Victor Y Pan. *Structured matrices and polynomials: unified superfast algorithms*. Springer Science & Business Media, 2012.
- Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. A critical look at the evaluation of GNNs under heterophily: Are we really making progress? In *International Conference on Learning Representations*, 2023.
- Michael Poli, Stefano Massaroli, Junyoung Park, Atsushi Yamashita, Hajime Asama, and Jinkyoo Park. Graph neural ordinary differential equations. *arXiv preprint arXiv:1911.07532*, 2019.
- T Konstantin Rusch, Michael M Bronstein, and Siddhartha Mishra. A survey on oversmoothing in graph neural networks. *SAM Research Report*, 2023, 2023.
- Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pages 618–626, 2017.
- Daniel A Spielman and Nikhil Srivastava. Graph sparsification by effective resistances. In *Proceedings of ACM symposium on Theory of computing*, pages 563–568, 2008.
- Kimberly Stachenfeld, Jonathan Godwin, and Peter Battaglia. Graph networks with spectral message passing. *arXiv preprint arXiv:2101.00079*, 2020.
- Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations*, 2022.
- Xiyuan Wang and Muhan Zhang. How powerful are spectral graph neural networks. In *International Conference on Machine Learning*, pages 23341–23362. PMLR, 2022.

Supplementary Materials

A. Further definitions and notations

This section formalizes conventions and assumptions that are used throughout the paper but may be implicit in the main exposition.

Secular equation

Secular equations yield the eigenvalues of the system after a rank-one update, based on the eigenvalues of the matrix being updated and the update itself.

Proposition A.1 (Secular equation, (Golub, 1973)). *Let $\mathbf{L} = \mathbf{U}\text{diag}(\boldsymbol{\lambda})\mathbf{U}^\top$ and $\tilde{\mathbf{L}} = \mathbf{L} + \rho \mathbf{v}\mathbf{v}^\top = \tilde{\mathbf{U}}\text{diag}(\tilde{\boldsymbol{\lambda}})\tilde{\mathbf{U}}^\top$. Then each $\tilde{\lambda}_j$ satisfies*

$$1 + \rho \sum_{i=1}^n z_i^2 / (\tilde{\lambda}_j - \lambda_i) = 0, \quad (11)$$

where $\mathbf{z} = \mathbf{U}^\top \mathbf{v}$.

In practice, we use Li’s secular equation solver (Li, 1994). Eigenvalues before and after the update satisfy an interleaving property (Golub, 1973).

Proposition A.2 (Eigenvalue interleaving (Bunch et al., 1978)). *Provided $\rho > 0$, the eigenvalues after the update satisfy:*

$$\lambda_1 \leq \tilde{\lambda}_1 \leq \lambda_2 \leq \dots \leq \lambda_n \leq \tilde{\lambda}_n. \quad (12)$$

This result guarantees that we can solve the secular equation efficiently (Gu and Eisenstat, 1996), and can be used to control the complexity of the matrix vector product (Fernández-Menduiña et al., 2025b) with Cauchy matrices.

Laplacian conventions

We consider weighted undirected graphs with adjacency matrix $\mathbf{W}$ and self-loop matrix $\mathbf{V} \succeq 0$. The generalized graph Laplacian (GGL) is defined as (Biyikoğlu et al., 2007):

$$\mathbf{L} \doteq \mathbf{D} - \mathbf{W} + \mathbf{V}, \quad (13)$$

where $\mathbf{D} = \text{diag}(\mathbf{1}^\top \mathbf{W})$ is the degree matrix. Unless otherwise stated, all theoretical results are derived for the combinatorial Laplacian $\mathbf{L}$, but a similar derivation can be applied to the normalized Laplacian:

$$\mathbf{L}_{\text{norm}} \doteq \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2}, \quad (14)$$

whose eigenvectors are orthonormal under the standard Euclidean inner product. The Cauchy factorization applies identically after normalization, up to a change of basis induced by $\mathbf{D}^{-1/2}$.

Eigenvalue ordering and degeneracies

Let $\mathbf{L} = \mathbf{U}\text{diag}(\boldsymbol{\lambda})\mathbf{U}^\top$ denote an eigendecomposition of a symmetric Laplacian matrix. Eigenvalues are assumed to be sorted in non-decreasing order. In the presence of repeated eigenvalues, the corresponding eigenvectors are not unique; any orthonormal basis spanning the eigenspace is valid.

When rank-one updates leave certain eigenspaces invariant, we apply deflation (Appendix D) to isolate the affected subspace. This operation preserves orthogonality and does not alter the asymptotic complexity of the factorization.

Affected spectral indices

Given a rank-one update $\tilde{\mathbf{L}} = \mathbf{L} + \rho \mathbf{v} \mathbf{v}^\top$ with $\mathbf{L} = \mathbf{U} \text{diag}(\boldsymbol{\lambda}) \mathbf{U}^\top$, define the projection vector

$$\mathbf{z} = \mathbf{U}^\top \mathbf{v}. \quad (15)$$

When $\mathbf{L}$ has distinct eigenvalues, we define the set of affected spectral indices as

$$\mathcal{S} \doteq \{i \mid z_i \neq 0\}. \quad (16)$$

When $\mathbf{L}$ has repeated eigenvalues, the associated eigenspaces are not uniquely defined. In this case, we exploit the rotational freedom within each eigenspace and choose an orthonormal eigenbasis such that, for each eigenspace, at most one basis vector has a nonzero projection onto $\mathbf{v}$ (Bunch et al., 1978). This choice is always possible and leaves the eigendecomposition of $\mathbf{L}$ unchanged. The definition of $\mathcal{S}$ then applies unchanged, with $\mathbf{U}$ denoting the chosen eigenbasis.

Only eigenpairs indexed by $\mathcal{S}$ are modified by the rank-one update; eigenvectors corresponding to indices outside $\mathcal{S}$ remain invariant. For bridge edges connecting two subgraphs, $\mathcal{S}$ is restricted to spectral components associated with those subgraphs, yielding localized Cauchy factors.

Rank-one versus rank- k updates

Each individual edge insertion corresponds to a rank-one update of the Laplacian. Cuts consisting of k bridge edges therefore induce a sequence of k rank-one updates, each associated with its own Cauchy factor. The resulting transformation is given by the product of these factors, yielding a rank- k modification of the spectrum.

All theoretical results are stated for rank-one updates and extend directly to rank- k cuts by composition and induction.

Spectral locality

We say that a linear operator $\mathbf{A} \in \mathbb{R}^{n \times n}$ is s -local in the spectral domain if it acts non-trivially on at most s spectral coordinates, i.e., if

$$\mathbf{A} = \mathbf{P}^\top \begin{bmatrix} \mathbf{I}_{n-s} & \mathbf{0} \\ \mathbf{0} & \mathbf{B} \end{bmatrix} \mathbf{P}, \quad (17)$$

for some permutation matrix $\mathbf{P}$ and $\mathbf{B} \in \mathbb{R}^{s \times s}$.

Cauchy factors induced by bridge edges are $O(|V_i| + |V_j|)$ -local, where V_i and V_j are the vertex sets of the connected subgraphs. In contrast, the full GFT basis is n -local. This distinction underpins the locality–globality trade-off exploited by L2G-Net.

Non-uniqueness of hierarchical decompositions

The hierarchical graph family (HGF) decomposition of a graph is generally not unique. Different choices of partitions and merge orders yield different factorizations with identical spectral semantics. Our objective is not to recover a unique hierarchy, but to identify decompositions that minimize computational cost while preserving spectral equivalence.

Graph sparsification

Graph sparsification seeks to approximate a graph $\mathcal{G}$ with a sparser graph $\tilde{\mathcal{G}}$ whose Laplacian remains spectrally close to that of $\mathcal{G}$. Let $\mathbf{L}$ and $\tilde{\mathbf{L}}$ denote the corresponding graph Laplacians. A $(1 \pm \varepsilon)$ spectral sparsifier satisfies

$$(1 - \varepsilon) \mathbf{x}^\top \mathbf{L} \mathbf{x} \leq \mathbf{x}^\top \tilde{\mathbf{L}} \mathbf{x} \leq (1 + \varepsilon) \mathbf{x}^\top \mathbf{L} \mathbf{x}, \quad \forall \mathbf{x} \in \mathbb{R}^n.$$

such a sparsifier can be constructed by sampling edges proportionally to their effective resistances, yielding $\tilde{\mathcal{G}}$ with $O(n \log n / \varepsilon^2)$ edges and preserving the spectrum of $\mathbf{L}$ up to relative error (Spielman and Srivastava, 2008). In this work, we use spectral sparsification in a localized manner, applying these guarantees to inter-subgraph cuts. This reduces the number of connecting edges while preserving the quadratic form of the global Laplacian.

B. Related work

Graph rewiring and transformers. Several approaches address long-range dependencies by modifying the graph structure or adding attention-based mechanisms. Graph rewiring methods aim to reduce the effective graph diameter (Barbero et al., 2023; Gasteiger et al., 2018; Giraldo et al., 2023), while graph transformers model global interactions through attention mechanisms (Deng et al., 2024). However, these approaches often add computational complexity by creating denser graph shift operators or weakening the graph’s topological inductive bias (Hariri et al., 2026; Giraldo et al., 2023) (cf. Section 6.5). In contrast, our Cauchy factorization is exact; potential approximations (spectral sparsification) simplify the computational complexity while providing guarantees on the spectrum of the resulting Laplacian (Spielman and Srivastava, 2008).

Positional encodings (PEs). Using Laplacian eigenvectors as node features has been proposed to inject global structural information into MPNNs (Dwivedi and Bresson, 2021; Huang et al., 2024). PEs enable global interactions, but they do so by allowing information in nodes that are far away (in the graph) to be combined. In contrast, L2G-Net enables long-range operations by composing local spectral operators across the hierarchy (subgraph-to-subgraph). These mechanisms are complementary: our work shows that combining L2G-Net with PEs further improves performance. PEs are typically identified with spectral methods, while we use spectral methods to identify subgraphs and create a hierarchical, graph-adaptive, task-independent architecture.

Deep GNNs and differential equations. Deep architectures (Li et al., 2019) or Neural ODEs (Poli et al., 2019; Chamberlain et al., 2021) can also help expand the receptive field via techniques like residual connections (Chen et al., 2020). Nonetheless, these methods still rely on local message passing to propagate information; so the receptive field grows at most linearly with depth, often requiring hundreds of layers to capture long-range interactions. In contrast, our hierarchical spectral framework decouples the receptive field from depth. This allows the architecture to focus on learning complex feature transformations without the signal degradation or over-smoothing issues (Rusch et al., 2023) typical of vertex-domain propagation.

Hybrid Spatial–Spectral GNNs. (Stachenfeld et al., 2020) compute a truncated GFT and learn a MPNN in a fully connected “spectral graph” whose vertices correspond to the GFT coefficients. This approach relies on partial eigendecomposition and operates on a fixed set of low-frequency components. Similarly, (Geisler et al., 2024) interleave vertex-domain polynomial message passing with truncated spectral filters, combining spatial operators with projections onto a limited set of eigenvectors. In both cases, spectral processing is confined to a subset of the spectrum. In contrast, L2G-Net neither truncates the spectrum nor approximates the graph Fourier transform. We show that the full GFT admits an exact hierarchical factorization via Cauchy matrices associated with interface edges. This enables global spectral processing without computing a dense eigendecomposition. Locality in L2G-Net is induced by graph partitioning and merging, rather than through polynomial filtering in the vertex domain.

Divide-and-conquer eigensolvers. The structure of eigenvector matrices arising from rank-one updates of symmetric eigenproblems was first analyzed by (Bunch et al., 1978). Divide and conquer eigensolvers were first proposed in (Cuppen, 1980; Gu and Eisenstat, 1995). This line of work has since been extended beyond tridiagonals to dense matrices admitting hierarchically semiseparable (HSS) representations (Ou and Xia, 2022), where the input matrix must first be compressed into HSS form by finding low-rank off-diagonal blocks, which are often approximated to a user-specified tolerance. Our setting differs in both its input and its output. First, the input is a sparse symmetric matrix specified by a graph, so hierarchical structure is given by the graph itself, making the resulting factorization exact rather than approximate. As a result, our complexity is stated in terms of a combinatorial property of the graph (its cut size) rather than a numerical off-diagonal rank. Second, the output of our method is not the eigendecomposition; instead, we obtain a Cauchy factorization of the eigenbasis, which is a structural statement about the GFT basis. In this factorization, each Cauchy factor corresponds to a specific bridge edge between subgraphs, decomposing the basis as a product of mixing operators between identifiable graph regions. This is what enables L2G-Net’s local to global architecture, where learnable filters slot in between Cauchy factors corresponding to specific cuts.

Fast algorithms for GFTs. Early works proposed approximating the Laplacian (Le Magoarou et al., 2018) or its eigenspace (Frerix and Bruna, 2019) using sequences of sparse or structured operators, such as Givens rotations or butterfly-like factorizations, aiming to reduce the complexity relative to dense eigendecomposition. However, spectral-mismatch errors with these methods are difficult to control *a priori*. While (Fernández-Menduiña et al., 2025a) established the link between rank-one updates and spectral graph theory, the resulting factorization of (3) was valid only under some restrictions on the eigenvalues of $\mathbf{L}$ and on the updates. Moreover, the work (Fernández-Menduiña et al., 2025a) focused on incremen-

Table 8. Dataset-specific hyperparameter settings for L2G-Net.

Dataset	d	Layers	Levels	K	Dropout	LR	Steps	Sparsif.
<i>Roman-empire</i>	64	5	1	8	0.25	2×10^{-4}	1000	0.005
<i>Amazon-ratings</i>	256	4	1	6	0.30	4×10^{-3}	1500	0.005
<i>Minesweeper</i>	32	14	1	6	0.225	2×10^{-3}	1000	0.005
<i>Tolokers</i>	64	3	1	4	0.20	8×10^{-4}	2000	0.01
<i>peptides-func</i>	256	4	1	12	0.35	1×10^{-3}	500	1
<i>peptides-struct</i>	256	4	1	12	0.3	1×10^{-3}	300	1
<i>Paris</i>	64	16	4	12	0.3	5×10^{-3}	5000	0.150
<i>Shanghai</i>	64	16	4	12	0.35	5×10^{-3}	5000	0.130
<i>Los Angeles</i>	64	16	5	12	0.25	2.5×10^{-3}	5000	0.100
<i>London</i>	64	16	6	12	0.4	2×10^{-3}	5000	0.125

tal rank-one updates of path graphs and did not consider the problem of decomposing an arbitrary graph to facilitate the factorization of its GFT.

C. Experimental details

The hyperparameters reported in Table 8 specify the architectural, spectral, and optimization settings used for L2G-Net across datasets.

The hidden dimension d denotes the dimensionality of node embeddings throughout the network. *Layers* corresponds to the number of stacked blocks, each consisting of a spectral filtering stage followed by a shared feed-forward module with residual connections.

The parameter K denotes the number of spectral filters per filter bank, which determines the resolution at which the graph spectrum is sampled. When dealing with inductive tasks, since the orientation of the graphs is arbitrary, the banks for the same level of hierarchy are shared across all subgraphs. For all datasets, filters are expressed using B-spline bases defined over the normalized Laplacian eigenvalues, which encourage smooth spectral responses.

Dropout denotes the feature dropout probability applied to node representations during training. *LR* is the base learning rate used by the AdamW optimizer. *Steps* denotes the total number of optimization steps performed during training.

Finally, *Sparsif.* refers to the cut sparsification ratio used during graph partitioning. In particular, in practice, we can interpret the parameter as using $\max(1, \text{edges after}/\text{edges before} \times 100)$ edges. This parameter controls the fraction of cut edges retained via effective resistance sampling, trading off computational cost and approximation accuracy in the spectral decomposition. A higher value retains more edges across the partition boundary, while smaller values lead to more aggressive sparsification.

Cross-partition edges are sparsified using resistance-based sampling. The partition is obtained via a Fiedler vector computed with LOBPCG ($k=2$, 40 iterations), using a quantile sweep over $[0.45, 0.55]$. Effective resistances are approximated with a JL (Jacques, 2015) with dimension $k = \lceil 24 \log |V| / \varepsilon^2 \rceil$ using $\varepsilon = 0.5$, with a minimum of $k = 20$. Crossing edges are sampled with replacement according to probabilities proportional to $w_e \tilde{R}_e$, where $\tilde{R}_e$ is the approximate resistance. A fraction ρ of crossing edges is retained, depending on the dataset, and sampled edges are reweighted by $(qp_e)^{-1}$. For details regarding the performance of the sparsifier, we refer the reader to (Spielman and Srivastava, 2008).

C.1. Base layer formulation

Following (Hariri et al., 2026), we adopt an ODE-inspired formulation. Let $\mathbf{X}(t)$ denote the node features at continuous time t . We let $\mathbf{U}(\Phi)$ be the local to global forward transform with local filtering in our method. Then,

$$d\mathbf{X}(t) = \mathbf{U}g_\theta(\boldsymbol{\lambda})\mathbf{U}(\Phi)^\top \mathbf{X}(t)\mathbf{W} dt. \quad (18)$$

Applying an explicit Euler discretization with step size $\epsilon > 0$ yields the update for layer $m = 0, \dots, M - 1$:

$$\mathbf{X}_{m+1} = \mathbf{X}_m + \epsilon \mathbf{U}g_\theta(\boldsymbol{\lambda})\mathbf{U}(\Phi)^\top \mathbf{X}_m \mathbf{W}. \quad (19)$$

Table 9. Statistics of real-world heterophilous graph datasets from Platonov *et al.* (Platonov *et al.*, 2023).

Dataset	Nodes	Edges	Avg. Degree	Features	Classes	k after sparse
<i>Roman-empire</i>	22,662	32,927	2.91	300	18	1
<i>Amazon-ratings</i>	24,492	93,050	7.60	300	5	1
<i>Minesweeper</i>	10,000	39,402	7.88	7	2	1
<i>Tolokers</i>	11,758	519,000	88.28	10	2	25

Depth allows for compositionality (learning complex non-linearities) rather than to expand the receptive field, which is already global due to the spectral formulation. Unlike (Hariri *et al.*, 2026), the complexity of the spectral filter does not directly control the norm of the layer Jacobian in our model. To reduce the parameter count, we share $\mathbf{W}$ and the spectral filter parameters θ, Φ across layers. Furthermore, we also share the spectral filter across channels.

C.2. Real-world graph properties

We provide some information about the properties of the graphs we consider in Table 9. We also include the cut size after sparsification during our experiments.

D. Deflation

The derivation in (Fernández-Menduiña *et al.*, 2025a), which is generalized in our work, assumes a worst-case scenario¹ where 1) the GGL before the rank-one update has no repeated eigenvalues and 2) the rank-one update used to add an edge is not orthogonal to any element in the original bases of the eigenspace. In these cases, we first deflate the matrix, reducing the computational complexity of the factorization. Following (Bunch *et al.*, 1978), consider the rank-one update:

$$\tilde{\mathbf{L}} = \mathbf{L} + \rho \mathbf{v} \mathbf{v}^\top = \tilde{\mathbf{U}} \text{diag}(\tilde{\boldsymbol{\lambda}}) \tilde{\mathbf{U}}^\top \quad (20)$$

where $\mathbf{L} = \mathbf{U} \text{diag}(\boldsymbol{\lambda}) \mathbf{U}^\top$ is the original eigendecomposition. Let $\mathbf{z} = \mathbf{U}^\top \mathbf{v}$ be the projection of the perturbation vector onto the eigenspace of $\mathbf{L}$. Then, we can write: $\tilde{\mathbf{L}} = \mathbf{U}(\text{diag}(\boldsymbol{\lambda}) + \rho \mathbf{z} \mathbf{z}^\top) \mathbf{U}^\top$. The idea of deflation is that, in some scenarios, we can find a set of eigenvectors of $\mathbf{L}$ (potentially different than $\mathbf{U}$) such that some of its elements are orthogonal to the original rank-one update $\mathbf{v}$, so that $\mathbf{z}$ has some zero entries. Since $\mathbf{U}$ in our case is given by our algorithm, we differentiate two cases: direct deflation (case 1) and deflation after applying a Householder reflector (case 2).

Case 1: Zero components ($z_i = 0$, for $i = 1, \dots, \ell$). The perturbation is then orthogonal to the basis of the original eigenspace. We obtain $\tilde{\lambda}_i = \lambda_i$ and $\tilde{\mathbf{u}}_i = \mathbf{u}_i$:

$$\tilde{\mathbf{L}} \mathbf{u}_i = \mathbf{U}(\text{diag}(\boldsymbol{\lambda}) + \rho \mathbf{z} \mathbf{z}^\top) \mathbf{e}_i = \mathbf{U}(\text{diag}(\boldsymbol{\lambda}) \mathbf{e}_i) = \lambda_i \mathbf{u}_i, \quad (21)$$

where $\mathbf{e}_i$ is the i th element of the canonical basis.

Deflation: Remove column and row i from $(\text{diag}(\boldsymbol{\lambda}) + \rho \mathbf{z} \mathbf{z}^\top)$. Construct the Cauchy matrix based on the updated secular equation. During multiplication, let $\mathbf{u}_i^\top \mathbf{x}$ be the i th coefficient, i.e., let it pass through unmodified.

Effect on complexity: Reduces the problem size from n to $n - \ell$, where ℓ is the number of zero components.

Case 2: Repeated eigenvalues. Consider a repeated eigenvalue λ_k with multiplicity m , and let $\mathcal{S}_k = \{i : \lambda_i = \lambda_k\}$ denote the index set of all eigenvectors corresponding to λ_k . The basis for this eigenspace is not unique; any orthogonal transformation of $\{\mathbf{u}_i\}_{i \in \mathcal{S}_k}$ yields another valid basis. We can exploit this freedom to choose a new basis that zeroes out components of $\mathbf{z}$. Let $\mathbf{z}_k$ be the m -dimensional subvector of $\mathbf{z}$ corresponding to the indices in $\mathcal{S}_k$. We can find an $m \times m$ orthogonal matrix $\mathbf{Q}$ (e.g., a Householder reflector) such that:

$$\mathbf{Q}^\top \mathbf{z}_k = \|\mathbf{z}_k\|_2 \mathbf{e}_1. \quad (22)$$

Applying this rotation to the basis of the eigenspace results in a new set of eigenvectors where the transformed perturbation vector, $\hat{\mathbf{z}}$, has $m - 1$ zero components in the subspace. Since the eigenvalues are identical within this subspace ($\lambda_i =$

¹Worst case in terms of complexity: applying deflation simplifies the factorization.

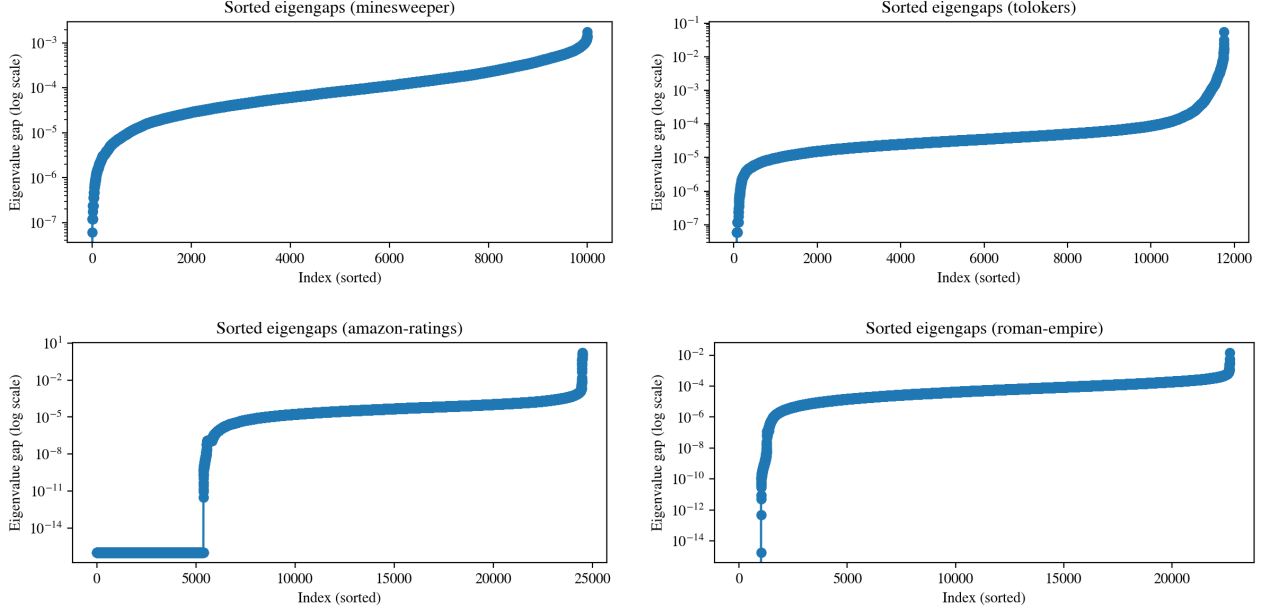


Figure 7. Sorted eigengaps of the normalized Laplacian on the heterophilous benchmarks of (Platonov et al., 2023). *Amazon-ratings* exhibits a large plateau of numerically zero gaps (approximately 20% of eigenvalues), and *Roman-empire* shows a smaller cluster of gaps below 10^{-12} ; *Minesweeper* and *Tolokers* are well-separated throughout. Deflation handles near-degenerate eigenvalues in all cases without affecting downstream accuracy.

$\lambda_k \forall i \in \mathcal{S}_k$), the diagonal matrix $\text{diag}(\boldsymbol{\lambda})$ is invariant under this rotation. This effectively reduces the problem to Case 1 for $m - 1$ eigenpairs, as they become orthogonal to the perturbation.

Deflation: Construct an orthogonal matrix $\mathbf{Q}$ that transforms the subvector $\mathbf{z}_k$ as shown above. Let $\hat{\mathbf{U}} = \mathbf{U}_{\mathcal{S}_k} \mathbf{Q}$. This rotation leaves $m - 1$ eigenpairs, $(\lambda_k, \hat{\mathbf{u}}_j)$, unaffected by the rank-one update. We can then remove the $m - 1$ rows and columns of $(\text{diag}(\boldsymbol{\lambda}) + \rho \hat{\mathbf{z}} \hat{\mathbf{z}}^\top)$ corresponding to the positions where $\hat{\mathbf{z}}$ is zero and solve the smaller problem.

Effect on complexity: Reduces the problem size from n to $n - (m - 1)$.

Near-degenerate eigenvalues are common in practice: for instance, on *Roman Empire*, about 4.5% of eigengaps fall below our threshold of 10^{-12} . Deflation handles these cases without degrading factorization accuracy or downstream performance. We show the eigengaps (the difference between consecutive eigenvalues) for the four datasets in (Platonov et al., 2023) in Figure 7.

E. Proof of Theorem 3.1

We show that the Laplacian eigenvectors of any graph in $\mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$ can be written as a chain product of CF. We denote by $\mathbf{x}_i$ the subset of $\mathbf{x}$ corresponding to the subset of nodes of $\mathcal{G}$ represented by $\mathcal{G}_i$.

E.1. CF decomposition

First, we establish the following property of any GGL, which states that the Laplacian decomposes into rank-one updates, one for each edge contribution.

Proposition E.1 ((Batson et al., 2014)). *Let $\mathbf{e}_j$ be the j th canonical vector, for $j = 1, \dots, n$. Then, the GGL $\mathbf{L}$ of an undirected graph can be written as*

$$\mathbf{L} = \sum_{(i,j) \in \mathcal{E}, i \neq j} w_{ij} (\mathbf{e}_i - \mathbf{e}_j)(\mathbf{e}_i - \mathbf{e}_j)^\top + \sum_{(i,i) \in \mathcal{E}} w_{ii} \mathbf{e}_i \mathbf{e}_i^\top, \quad (23)$$

where w_{ij} denotes the edge weights, for $i, j = 1, \dots, |\mathcal{V}|$.

We can state now the following result, which generalizes (Fernández-Menduiña et al., 2025a) to arbitrary symmetric matrices.

Theorem E.1 (Progressive decomposition identity). *The eigenvector basis $\tilde{\mathbf{U}}^\top$ of $\tilde{\mathbf{L}} = \tilde{\mathbf{U}}\text{diag}(\tilde{\boldsymbol{\lambda}})\tilde{\mathbf{U}}^\top = \mathbf{L} + \alpha\mathbf{v}\mathbf{v}^\top$, with $\mathbf{L} = \mathbf{U}\text{diag}(\boldsymbol{\lambda})\mathbf{U}^\top$, decomposes as*

$$\tilde{\mathbf{U}}^\top = -\mathbf{D}(\tilde{\boldsymbol{\lambda}}, \boldsymbol{\lambda})\mathbf{U}^\top. \quad (24)$$

Proof. When $\mathcal{S}$ is empty, the result boils down to the progressive factorization in (Fernández-Menduiña et al., 2025a). When $\mathcal{S}$ is not empty, we apply deflation first, and construct $\mathbf{C}(\tilde{\boldsymbol{\lambda}}_{\mathcal{S}}, \boldsymbol{\lambda}_{\mathcal{S}})$ from the deflated matrix. $\square$

When $\tilde{\mathbf{L}}$ is the Laplacian of a graph, this result offers a *progressive factorization* of the Laplacian eigenvectors: we can decompose the new basis as the eigenvectors of the unperturbed Laplacian multiplied by a CF.

Then, we can design a decomposition that starts with a fully disconnected graph and progressively adds edges. Let $\boldsymbol{\lambda}_0, \dots, \boldsymbol{\lambda}_{|\mathcal{E}|}$ be the sequence of eigenvalues of the GGL of the graph obtained by performing each step of this process. Then, we can state the following result.

Lemma E.1 (CF decomposition). *The Laplacian eigenvectors of the GGL $\mathbf{L} = \mathbf{U}\text{diag}(\boldsymbol{\lambda})\mathbf{U}^\top$ of any graph $\mathcal{G}$ can be written as the product of $|\mathcal{E}|$ CF:*

$$\mathbf{U}^\top = \mathbf{D}(\boldsymbol{\lambda}_{|\mathcal{E}|}, \boldsymbol{\lambda}_{|\mathcal{E}|-1}) \dots \mathbf{D}(\boldsymbol{\lambda}_2, \boldsymbol{\lambda}_1)\mathbf{D}(\boldsymbol{\lambda}_1, \boldsymbol{\lambda}_0). \quad (25)$$

Proof. The result follows from applying Theorem E.1 accounting for every possible rank-one update in Proposition 2.1. $\square$

Now, we can use the previous result to state a decomposition for graphs in $\mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$. We define the stack of matrices of base transforms as:

$$\mathbf{U}_b^\top \doteq \begin{bmatrix} \mathbf{U}_1^\top & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{U}_2^\top & \dots & \mathbf{0} \\ \vdots & \ddots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{U}_m^\top \end{bmatrix}. \quad (26)$$

Let $\boldsymbol{\lambda}$ be the eigenvalues of the GGL for the graph with the isolated base components. We then denote by $\tilde{\boldsymbol{\lambda}}_i$, for $i = 1, \dots, k(2^L - 1)$, the eigenvalues resulting from each rank-one update.

Lemma E.2 (HGF decomposition). *Let $\tilde{\mathbf{U}}$ be Laplacian eigenvectors of any graph $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$. Then,*

$$\tilde{\mathbf{U}}^\top = \mathbf{D}(\tilde{\boldsymbol{\lambda}}_{k(2^L-1)}, \tilde{\boldsymbol{\lambda}}_{k(2^L-1)-1}) \dots \mathbf{D}(\tilde{\boldsymbol{\lambda}}_1, \boldsymbol{\lambda})\mathbf{U}_b^\top. \quad (27)$$

Proof. The result follows by induction from Proposition 2.1 and Theorem E.1. $\square$

Although the decomposition consists of $k(2^L - 1)$ products, in a HGF each CF modifies only a reduced subset of the input vector, which can be exploited to systematically reduce complexity. The next result shows that CF are localized only to the subgraphs they modify.

Proposition E.2 (Edge connection). *Consider a graph $\mathcal{G}$ with GGL $\mathbf{L}$ formed by a set of m disconnected subgraphs $\mathcal{G}_i$, for $i = 1, \dots, m$. Let $\tilde{\mathbf{L}} = \mathbf{L} + \rho\mathbf{v}\mathbf{v}^\top$ and $n = |\mathcal{V}|$. Then,*

$$\mathbf{D}(\tilde{\boldsymbol{\lambda}}, \boldsymbol{\lambda}) = \mathbf{P}_S^\top \begin{bmatrix} \mathbf{I}_{n-s \times n-s} & \mathbf{0} \\ \mathbf{0} & -\mathbf{C}(\tilde{\boldsymbol{\lambda}}_{\mathcal{S}}, \boldsymbol{\lambda}_{\mathcal{S}}) \end{bmatrix} \mathbf{P}_S \in \mathbb{R}^{n \times n}. \quad (28)$$

with

- $s = |\mathcal{V}_i| + |\mathcal{V}_j|$ when the rank-one update connects $\mathcal{G}_i$ to $\mathcal{G}_j$, with $i \neq j$,
- $s = |\mathcal{V}_i|$ when the rank-one update affects only the subgraph $\mathcal{G}_i$.

Proof. From the progressive property [Theorem E.1](#) and the definition of CF, we have to show that the transform version of the rank-one update $\mathbf{U}_b^\top \mathbf{v}$ has at least $n - s$ zeros. Since

$$\mathbf{z} = \mathbf{U}_b^\top \mathbf{v} = \begin{bmatrix} \mathbf{U}_1^\top & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{U}_2^\top & \cdots & \mathbf{0} \\ \vdots & \ddots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{U}_m^\top \end{bmatrix} \mathbf{v} = \begin{bmatrix} \mathbf{U}_1^\top \mathbf{v}_1 \\ \mathbf{U}_2^\top \mathbf{v}_2 \\ \vdots \\ \mathbf{U}_m^\top \mathbf{v}_m \end{bmatrix}, \quad (29)$$

where $\mathbf{v}_i$ denotes the subsection of the vector $\mathbf{v}$ that corresponds to the i th subgraph. Now, for the first case, $\mathbf{v}$ only has two non-zero components, which will be in $\mathbf{v}_i$ and $\mathbf{v}_j$. Therefore, $\mathbf{U}_k^\top \mathbf{v}_k$ is zero for all $k \neq i, j$. This set comprises $n - s$ components, with $s = |\mathcal{V}_i| + |\mathcal{V}_j|$. Therefore, the first result follows.

For the second case, both non-zero components will lie in $\mathbf{v}_i$. Therefore, $\mathbf{U}_k^\top \mathbf{v}_k$ is zero for all $k \neq i$. This set comprises $n - s$ components, with $s = |\mathcal{V}_i|$. Therefore, the result follows. $\square$

The previous result states that when we join two distinct subgraphs $\mathcal{G}_i$ and $\mathcal{G}_j$ via a rank-one update, the corresponding CF is an identity matrix except for a block of size $s = |\mathcal{V}_i| + |\mathcal{V}_j|$. This localizes the update to only the dimensions corresponding to the vertices of those two subgraphs. In the next section, we provide a method to implement these structured matrix products efficiently.

F. Constructing the factorization

We detail our algorithm for eigendecomposition in [Algorithm 3](#). We analyze its complexity as follows:

- **Apply transform.** Since we add k edges between pairs of subgraphs, the maximum number of edges touching any given subgraph is kL . Therefore, for a matrix $\mathbf{M} \in \mathbb{R}^{s \times s}$, the complexity of each call is

$$O(g(s) k L),$$

where $g(s)$ is the complexity of the matrix-vector product with the input matrix. This bound is a worst-case bound over the full hierarchy; tighter level-wise bounds are used in the merging analysis

- **Initialization.** Creating and updating each element of $\mathcal{Z}$ is linear in the size of the smallest graphs, $n/2^L$. Since we have at most $k \sum_{\ell=1}^L 2^{L-\ell} = k(2^L - 1)$ elements in $\mathcal{Z}$, the cost is upper bounded by $O(nk)$.

The eigendecomposition of the leaf graphs has cost

$$O\left(\sum_{i=1}^m f_i(n)\right).$$

Updating all elements of $\mathcal{Z}$ across all levels requires

$$k(2^L - 1)O(n/2^L) = O(kn).$$

While we are multiplying by dense matrices, multiplying by each element of the canonical basis amounts to selecting the corresponding column of the dense matrix. Since we have to scale by the weight, the complexity becomes $2n/2^L$, which yields the result.

- **Hierarchical merging.** Solving the secular equation at level ℓ costs $O((n/2^{L-\ell})^2)$. Since at that level, at most $k 2^{L-\ell}$ edges participate, this operation has complexity

$$\sum_{\ell=1}^L k 2^{L-\ell} O((n/2^{L-\ell})^2) = O(kn^2).$$

We must also apply the corresponding Cauchy updates. The complexity of an update at level ℓ is $O(g(n/2^{L-\ell}) k(L - \ell))$ per edge, since each edge update requires a multiplication by all the other incoming updates that touch the corresponding subgraph. Therefore the total merging complexity is

$$\sum_{\ell=1}^L k 2^{L-\ell} O(k(L - \ell)g(n/2^{L-\ell})) = O\left(k^2 \sum_{\ell=1}^L (L - \ell)g(n/2^{L-\ell})2^{L-\ell}\right).$$

Since $g(n) = O(n \log n)$ (Pan, 2012), we reach

$$O(kn^2 + k^2 n(L^2 \log n - L^3)).$$

If, instead, a $g(n) = O(n^2)$ algorithm is used, the complexity of the merging step becomes $O(k^2 n^2)$, with a quadratic relationship on k . In practice, even if we opt for the $g(x) = O(x^2)$ algorithm, we have observed that the complexity of the update is closer to $O((L - \ell)g(n/2^{L-\ell}))$ (since not all the edges to update lay always on the same subgraph), which yields again $O(kn^2)$ behavior.

Thus, the full complexity of the algorithm is

$$O\left(kn^2 + \sum_{i=1}^m f_i(n)\right)$$

Parallel solver. Under parallel execution, with m processors available, we focus on the parallel time $T_m(\cdot)$:

- **Initialization.** Since each block operates on disjoint edges in $\mathcal{Z}$, operations can be carried out in parallel. Assuming at least m processors are available, by Brent's theorem (JáJá, 1992), we can compute the eigendecomposition in time

$$T_m(n) = O\left(\max_{i=1, \dots, m} f_i(n)\right).$$

- **Hierarchical merging.** All subgraph pairs at level ℓ are independent, so the work at that level parallelizes fully. The work on each level is still governed by the cost of solving the secular equation, which has to be solved sequentially. Therefore, the time to compute this step remains in $T_m(n, k) = O(kn^2)$.

Thus, we can compute the parallel version in time

$$T_m(n, k) = O\left(kn^2 + \max_{i=1, \dots, m} f_i(n)\right).$$

G. Proof of Theorem 5.1

We establish the result in three steps: we first show that polynomial filters and global spectral filters coincide on finite graphs, then prove containment, and finally exhibit a strict separation.

G.1. Polynomial filters and global spectral filters coincide on finite graphs

A polynomial filter of degree K computes

$$\sum_{k=0}^K \alpha_k \mathbf{L}^k \mathbf{x} = \mathbf{U} p(\mathbf{\Lambda}) \mathbf{U}^\top \mathbf{x}, \quad (30)$$

where $p(\lambda) = \sum_k \alpha_k \lambda^k$. Conversely, any global spectral filter $\mathbf{U} g(\mathbf{\Lambda}) \mathbf{U}^\top$ for arbitrary $g(\cdot)$ can be expressed as a polynomial in $\mathbf{L}$ of degree at most $n - 1$. This follows from two observations:

1. **Cayley–Hamilton:** $\mathbf{L}$ satisfies its own characteristic polynomial of degree n , so the algebra generated by $\mathbf{L}$ is spanned by $\{\mathbf{I}, \mathbf{L}, \dots, \mathbf{L}^{n-1}\}$.

Algorithm 2 Solve secular Equation

Input:

- 1: Old eigenvalues $\lambda = (\lambda_1, \dots, \lambda_n)$, sorted ascending
- 2: Projection vector $\mathbf{z} \in \mathbb{R}^n$

Output:

- 3: Updated eigenvalues $\tilde{\lambda}$
- 4: Cauchy matrix $\mathbf{C} \in \mathbb{R}^{n \times n}$ (columns are eigenvectors)
- 5: **function** SOLVESECULAR($\lambda, \mathbf{z}$)
- 6: $n \leftarrow \text{length}(\lambda)$
- 7: Initialize $\tilde{\lambda} \in \mathbb{R}^n$
- 8: **for** $j = 1 \dots n$ **do**
- 9: Solve for $\tilde{\lambda}_j$ in $(\lambda_j, \lambda_{j+1})$: $1 + \sum_{k=1}^n z_k^2 / (\tilde{\lambda}_j - \lambda_k) = 0$
- 10: Initialize $\mathbf{C} \in \mathbb{R}^{n \times n}$
- 11: **for** $j = 1 \dots n$ **do** ▷ For each new eigenvalue (Column)
- 12: **for** $i = 1 \dots n$ **do** ▷ For each component (Row)
- 13: $C_{ij} \leftarrow z_i / (\tilde{\lambda}_j - \lambda_i)$
- 14: Normalize column j : $\mathbf{C}_{:,j} \leftarrow \mathbf{C}_{:,j} / \|\mathbf{C}_{:,j}\|_2$
- 15: **return** $\tilde{\lambda}, \mathbf{C}$

2. **Lagrange interpolation:** On a fixed graph with $m \leq n$ distinct eigenvalues, any function $g(\cdot)$ on the spectrum $\{\lambda_1, \dots, \lambda_m\}$ can be interpolated by a polynomial of degree at most $m - 1 \leq n - 1$ (Wang and Zhang, 2022).

Therefore, on a fixed finite graph, the two classes coincide:

$$\{\mathbf{U} g(\mathbf{A}) \mathbf{U}^\top : g(\cdot) \text{ arbitrary}\} = \{p(\mathbf{L}) : p(\cdot) \text{ polynomial of degree } \leq n - 1\}. \quad (31)$$

We use “ $g(\mathbf{L})$ ” to denote this class throughout.

G.2. Containment: $g(\mathbf{L}) \subseteq \text{L2G-Net}$

Consider a graph $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$. By Theorem 3.1, the GFT admits the Cauchy factorization

$$\mathbf{U}^\top = \mathbf{D}(\lambda, \tilde{\lambda}_{K-1}) \cdots \mathbf{D}(\tilde{\lambda}_1, \tilde{\lambda}_0) \mathbf{U}_0^\top, \quad (32)$$

where $\mathbf{U}_0 = \text{blkdiag}(\mathbf{U}_1, \dots, \mathbf{U}_m)$ collects the base subgraph GFTs and each $\mathbf{D}$ is a Cauchy factor.

In L2G-Net, at each hierarchical level r and subgraph pair p , a learnable spectral filter $g_{r,p}(\lambda_{r,p})$ is applied before the Cauchy merge. Setting all intermediate filters to the identity, i.e., $g_{r,p}(\lambda) = 1$ for all λ , all $r = 0, \dots, L - 1$, and all p , the forward transform $\mathbf{U}(\Phi)^\top$ reduces exactly to $\mathbf{U}^\top$ by Theorem 3.1. The L2G-Net output then becomes

$$\mathbf{X}_{\text{out}} = \mathbf{U} g_\theta(\mathbf{A}) \mathbf{U}^\top \mathbf{X}, \quad (33)$$

which is a standard global spectral filter. Since the identity is within the parameterization of L2G-Net’s local filters, every global spectral filter is realizable. $\square$

G.3. Strict containment: $g(\mathbf{L}) \subsetneq \text{L2G-Net}$

We show that L2G-Net can represent operators that no global spectral filter $g(\mathbf{L})$ can express, by exhibiting an explicit counterexample.

General necessary condition. Consider a graph $\mathcal{G} \in \mathcal{F}(1, \{\mathcal{G}_1, \mathcal{G}_2\}, k)$ with $k \geq 1$, i.e., two subgraphs connected by at least one bridge edge. With one level of hierarchy, the L2G-Net forward transform is

$$\mathbf{U}(\Phi)^\top = g_1(\mathbf{A}) \mathbf{D} \text{blkdiag}(g_{0,1}(\mathbf{A}_1), g_{0,2}(\mathbf{A}_2)) \mathbf{U}_0^\top, \quad (34)$$

where $\mathbf{D}$ is the product of Cauchy factors for the bridge edges and $g_1(\cdot)$ acts on the merged spectrum. Define

$$\mathbf{F}_{\text{loc}} \doteq \text{blkdiag}(g_{0,1}(\mathbf{\Lambda}_1), g_{0,2}(\mathbf{\Lambda}_2)). \quad (35)$$

The full L2G-Net operator is

$$\mathbf{T}_{\text{L2G}} = \mathbf{U} g_\theta(\mathbf{\Lambda}) g_1(\mathbf{\Lambda}) \mathbf{D} \mathbf{F}_{\text{loc}} \mathbf{U}_0^\top, \quad (36)$$

while a global spectral filter takes the form

$$\mathbf{T}_{\text{GFT}} = \mathbf{U} h(\mathbf{\Lambda}) \mathbf{U}^\top = \mathbf{U} h(\mathbf{\Lambda}) \mathbf{D} \mathbf{U}_0^\top, \quad (37)$$

where the last equality uses Theorem 3.1 with identity local filters.

For $\mathbf{T}_{\text{L2G}} = \mathbf{T}_{\text{GFT}}$ to hold for some $h(\cdot)$, we require

$$g_\theta(\mathbf{\Lambda}) g_1(\mathbf{\Lambda}) \mathbf{D} \mathbf{F}_{\text{loc}} = h(\mathbf{\Lambda}) \mathbf{D}. \quad (38)$$

Let $f(\mathbf{\Lambda}) := g_\theta(\mathbf{\Lambda}) g_1(\mathbf{\Lambda})$ (a diagonal matrix). Since $\mathbf{D}$ is orthogonal (being a product of orthogonal Cauchy-like matrices), right-multiplying by $\mathbf{D}^\top$ yields

$$f(\mathbf{\Lambda}) \mathbf{D} \mathbf{F}_{\text{loc}} \mathbf{D}^\top = h(\mathbf{\Lambda}). \quad (39)$$

Since $h(\mathbf{\Lambda})$ is diagonal, a necessary condition for the L2G-Net operator to collapse to a global spectral filter is

$$\mathbf{D} \mathbf{F}_{\text{loc}} \mathbf{D}^\top \text{ is diagonal.} \quad (\star)$$

Counterexample: two singleton subgraphs. Consider $n_1 = n_2 = 1$: two isolated nodes connected by a single edge of weight $w > 0$. The base subgraphs are singletons with trivial GFTs ($\mathbf{U}_0 = \mathbf{I}_2$). The Laplacian of the disconnected graph is $\mathbf{L}_0 = \mathbf{0}$, and adding the bridge edge gives

$$\mathbf{L} = w \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad (40)$$

with eigenvalues 0 and $2w$, and eigenvectors

$$\mathbf{U} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \quad (41)$$

By Theorem 3.1, $\mathbf{U}^\top = \mathbf{D} \cdot \mathbf{U}_0^\top = \mathbf{D} \cdot \mathbf{I}_2$, so the Cauchy factor is

$$\mathbf{D} = \mathbf{U}^\top = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \quad (42)$$

This is a nontrivial orthogonal matrix (neither diagonal nor a permutation). Now let $\mathbf{F}_{\text{loc}} = \text{diag}(a, b)$ with $a \neq b$. Then

$$\mathbf{D} \mathbf{F}_{\text{loc}} \mathbf{D}^\top = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} a+b & a-b \\ a-b & a+b \end{pmatrix}. \quad (43)$$

The off-diagonal entry is $\frac{a-b}{2} \neq 0$ whenever $a \neq b$. Therefore, condition $(\star)$ fails, and the L2G-Net operator $\mathbf{T}_{\text{L2G}}$ cannot be expressed as any global spectral filter $g(\mathbf{L})$.

Since this construction uses a valid member of $\mathcal{F}(1, \{\mathcal{G}_1, \mathcal{G}_2\}, 1)$ and a non-constant local filter (which is within L2G-Net’s parameterization), the inclusion is strict. $\square$

Remark G.1 (Generality beyond the counterexample). The two-node example is a minimal proof of strict inclusion. The phenomenon extends to arbitrary graphs: since Cauchy factors arising from bridge edges are dense orthogonal matrices with all nonzero entries, the conjugation $\mathbf{D} \mathbf{F}_{\text{loc}} \mathbf{D}^\top$ is non-diagonal whenever $\mathbf{F}_{\text{loc}}$ is not proportional to the identity.

H. L2G-Net intuition

Given two dense subgraphs connected by a bridge edge (e.g., barbell-like graph (Topping et al., 2022), as shown in Figure 8), MPNNs require many aggregation steps to share information between subgraphs, which may lead to oversquashing (Alon and Yahav, 2021). In L2G-Net, the graph is partitioned into subgraphs, which are first processed separately, and then the Cauchy factor operates across the bridge edge to mix information from the subgraphs, enabling long-range communication with a single layer. Unlike the GFT, where all spectral components are global, L2G-Net can learn different ways to combine information from subgraphs for each spectral component.

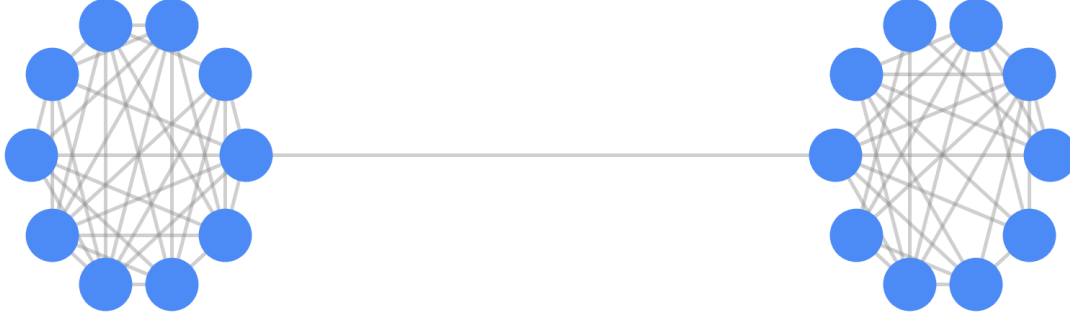


Figure 8. Example of a barbell-like graph.

Table 10. End-to-end training time (mm:ss) to convergence on heterophilous benchmarks (Platonov et al., 2023), under identical hardware.

Method	Mines.	Tolok.	Am. Rat.	R. Emp.
GCN	01:23	01:53	02:12	02:02
ChebNet	03:56	06:18	11:22	13:08
Polynormer	15:12	08:32	35:00	28:56
L2G-Net (ours)	08:40	03:04	34:56	50:03

I. More experimental results

In Table 12, we compare our method with a benchmark based on (Platonov et al., 2023). In Table 13 we compare the results in the peptides datasets of (Dwivedi et al., 2022b). In Table 14, we compare the results in the City-Networks dataset of (Liang et al., 2025).

I.1. Learned filters

We show the learned spectral filters for different graphs in (Platonov et al., 2023) (Figure 9). We consider only a two-level decomposition. We also add the histogram of the distribution of the eigenvalues in the background, for each subgraph and for the global graph. We observe that hierarchical processing adapts to the task, which contributes to the interpretability of our method.

I.2. GradCAM measurements

We show the GradCAM attribution curves for the three other graphs (*Tolokers*, *Roman Empire*, and *Amazon Ratings*) in (Platonov et al., 2023) in Figure 10. We observe that the same behavior we reported in Figure 6 generalizes to the other graphs in the suite.

I.3. Runtime Measurements

In this section we provide a detailed empirical comparison of the runtime of L2G-Net against representative baselines. All experiments are run under identical hardware and training protocols. For L2G-Net, GCN, and ChebNet we match architecture hyperparameters (depth, hidden dimension); for Polynormer we use the configuration provided by the original authors, consistent with the setup in Table 2. End-to-end training times for L2G-Net include the cost of computing the Cauchy factorization.

End-to-end training time. Table 10 reports the total wall-clock time required to train each method to convergence on the heterophilous benchmarks of (Platonov et al., 2023). Although Polynormer attains a lower per-epoch cost than L2G-Net (see Table 11), its two-stage training procedure requires substantially more epochs to converge, so its overall training time is longer than that of L2G-Net on three of the four datasets. Compared to ChebNet, L2G-Net incurs a moderate overhead

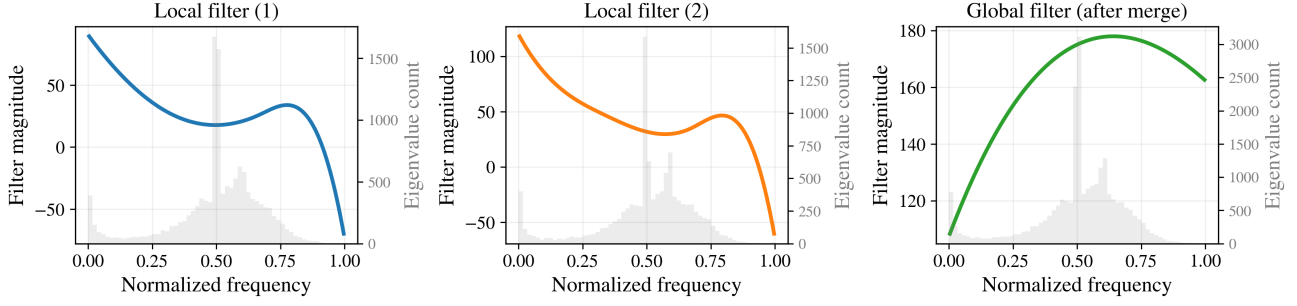
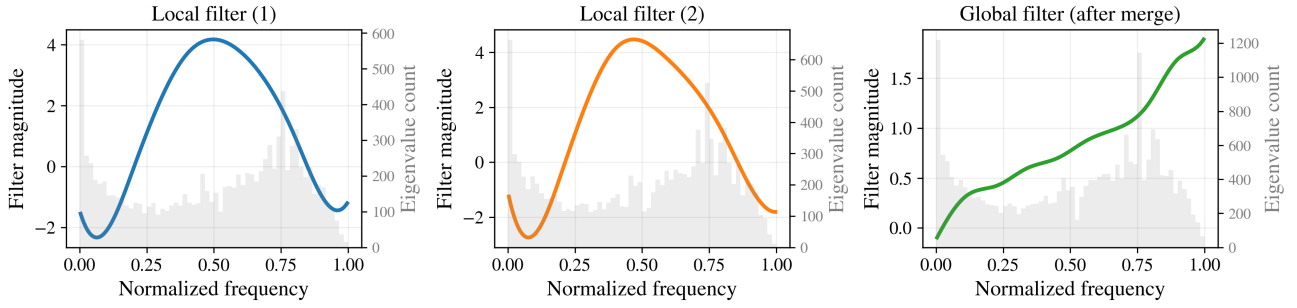
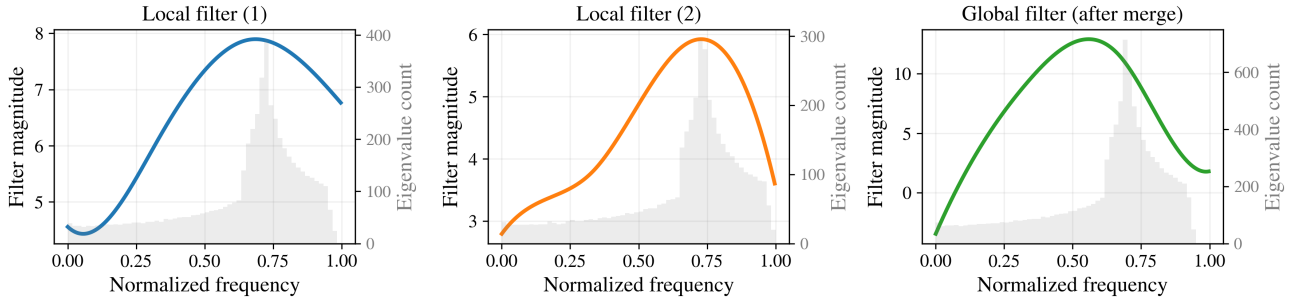
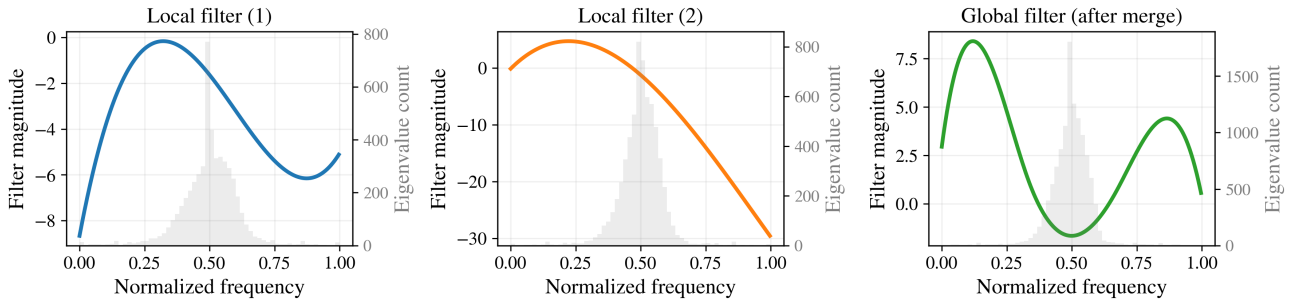
(a) *Amazon-ratings*

 (b) *Roman-empire*

 (c) *Minesweeper*

 (d) *Tolokers*


Figure 9. Learned spectral filters for different graphs in (Platonov et al., 2023). In tasks where both low- and high-frequency information are needed (e.g., *Tolokers*/*Amazon-ratings*), local filters emphasize low frequencies, while global filters focus on higher frequencies. When the signal lies in high frequencies (*Minesweeper*/*Roman-empire*), both local and global filters focus on high-frequency components. This indicates that the hierarchical processing adapts to the task.

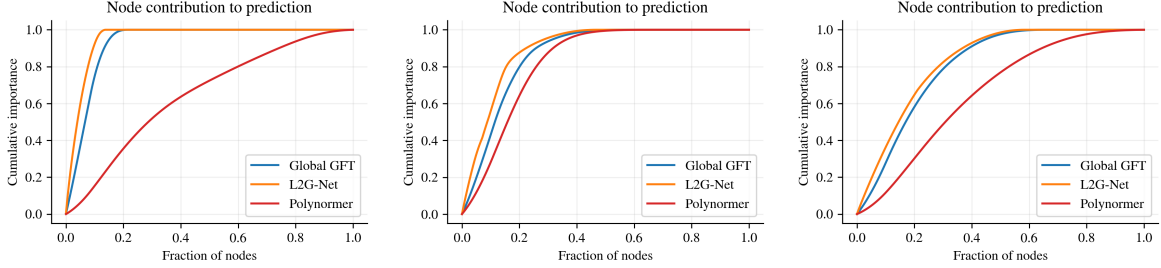


Figure 10. Cumulative node contribution to the prediction on *Tolokers*, *Roman Empire*, and *Amazon Ratings* (from left to right). Average across validation set; shaded areas indicate confidence intervals. L2G-Net consistently concentrates predictive importance on a smaller fraction of nodes than both the Global GFT and Polynormer, showing consistent behavior across datasets.

Table 11. Per-epoch wall-clock time (s/epoch) on heterophilous benchmarks (Platonov et al., 2023).

Method	Mines.	Tolok.	Am. Rat.	R. Emp.
GCN	0.05	0.06	0.07	0.06
ChebNet	0.12	0.21	0.33	0.40
Polynormer	0.29	0.18	0.56	0.48
L2G-Net (ours)	0.35	0.13	1.04	1.54

that reflects its richer subgraph-level spectral processing, but remains well within the same order of magnitude. Compared to a standard MPNN such as GCN, L2G-Net is slower; this is expected, as our method targets the regime of exact spectral processing rather than linear-time message passing.

Per-epoch wall-clock time. Table 11 reports per-epoch wall-clock times in seconds. GCN is the fastest, consistent with its linear $\mathcal{O}(|E|)$ complexity. ChebNet and Polynormer occupy an intermediate range. L2G-Net is competitive on smaller graphs (*Minesweeper*, *Tolokers*) but its per-epoch cost grows with n , reflecting the $\mathcal{O}(n^2)$ complexity of the Cauchy factorization-based forward pass. The trade-off is favorable in the end-to-end training picture because L2G-Net requires fewer epochs to converge than transformer baselines.

Algorithm 3 Cauchy factorization

Input:

- 1: Hierarchical graph $\mathcal{G} \in \mathcal{F}(L, \{\mathcal{G}_i\}_{i=1}^m, k)$
- 2: Bridge edge sets $\mathcal{E}_{\ell,i}$ with corresponding weights $w(\cdot)$ for $i = 1, \dots, 2^{L-\ell}$ and levels $\ell = 1 \dots L$.

Output:

- 3: Final eigenvalues $\lambda \in \mathbb{R}^N$, Cauchy rotation history $\mathcal{H} = \{\mathcal{H}_1, \dots, \mathcal{H}_L\}$.

```

4: function ApplyTransform( $\mathcal{I}, \mathbf{M}, \mathcal{Z}$ )
5:    $\mathcal{E}_{\text{act}} \leftarrow \{e = (u, v) \in \text{Keys}(\mathcal{Z}) \mid \{u, v\} \cap \mathcal{I} \neq \emptyset\}$  ▷ Update edge vectors touching node set  $\mathcal{I}$ 
6:   for each  $e \in \mathcal{E}_{\text{act}}$ 
7:      $\mathbf{z}_{\text{loc}} \leftarrow \text{Extract}(\mathcal{Z}[e], \mathcal{I})$ 
8:      $\mathcal{Z}[e] \leftarrow \text{Insert}(\mathcal{Z}[e], \mathcal{I}, \mathbf{M}^\top \mathbf{z}_{\text{loc}})$ 
9:   return  $\mathcal{Z}$ 

10: 1. Initialization step
11:  $\mathcal{E}_{\text{total}} \leftarrow \bigcup_{\ell,i} \mathcal{E}_{\ell,i}$ 
12: Initialize  $\mathcal{Z}$  as map: edge  $\mapsto$  sparse length- $N$  vector
13: for each  $e = (u, v) \in \mathcal{E}_{\text{total}}$  do
14:    $\mathcal{Z}[e] \leftarrow \sqrt{w(e)} (\mathbf{e}_u - \mathbf{e}_v)$ 
15:  $\mathcal{S}_{\text{curr}} \leftarrow$  empty list
16: for  $i = 1 \dots m$ 
17:    $(\mathbf{U}_i, \lambda_i) \leftarrow \text{DENSEEIGSLAP}(\mathcal{G}_i)$ 
18:    $\mathcal{I}_i \leftarrow$  sorted node set of leaf  $\mathcal{G}_i$ 
19:    $\mathcal{Z} \leftarrow \text{APPLYTRANSFORM}(\mathcal{I}_i, \mathbf{U}_i, \mathcal{Z})$ 
20:   Append  $(\lambda_i, \mathcal{I}_i)$  to  $\mathcal{S}_{\text{curr}}$ 

21: 2. Hierarchical merging
22: for  $\ell = 1 \dots L$  do
23:    $\mathcal{S}_{\text{next}} \leftarrow$  empty list
24:    $\mathcal{H}_\ell \leftarrow \emptyset$ 
25:   for  $i = 1 \dots |\mathcal{S}_{\text{curr}}|/2$  do
26:     Retrieve left  $(\lambda_A, \mathcal{I}_A) \leftarrow \mathcal{S}_{\text{curr}}[2i-1]$ 
27:     Retrieve right  $(\lambda_B, \mathcal{I}_B) \leftarrow \mathcal{S}_{\text{curr}}[2i]$ 
28:      $\mathcal{I}_{\text{new}} \leftarrow \mathcal{I}_A \cup \mathcal{I}_B$ 
29:      $\lambda_{\text{new}} \leftarrow [\lambda_A; \lambda_B]$ 
30:     for each bridge  $e \in \mathcal{E}_{\ell,k}$  do
31:        $\mathbf{z} \leftarrow \text{Extract}(\mathcal{Z}[e], \mathcal{I}_{\text{new}})$ 
32:        $(\lambda_{\text{new}}, \mathbf{C}) \leftarrow \text{SOLVESECULAR}(\lambda_{\text{new}}, \mathbf{z})$ 
33:        $\mathcal{Z} \leftarrow \text{APPLYTRANSFORM}(\mathcal{I}_{\text{new}}, \mathbf{C}, \mathcal{Z})$ 
34:        $\mathcal{H}_\ell \leftarrow \mathcal{H}_\ell \cup \{\mathbf{C}\}$ 
35:     Append  $(\lambda_{\text{new}}, \mathcal{I}_{\text{new}})$  to  $\mathcal{S}_{\text{next}}$ 
36:    $\mathcal{S}_{\text{curr}} \leftarrow \mathcal{S}_{\text{next}}$ 

37: return final  $\lambda$  from  $\mathcal{S}_{\text{curr}}$  and  $\mathcal{H}$ 

```

Table 12. Performance comparison on long-range benchmarks. Results are reported as mean $\pm$ standard deviation. We report Acc for *Roman-empire* and *Amazon-ratings* and AUC for *Tolokers* and *Minesweeper*. Higher is better ($\uparrow$). Best method appears in boldface, second best method underlined, third best method in italics.

Model	Roman-empire	Amazon-ratings	Minesweeper	Tolokers
<i>MPNNs</i>				
GAT	80.87 $\pm$ 0.30	49.09 $\pm$ 0.63	92.01 $\pm$ 0.68	83.70 $\pm$ 0.47
GAT (LapPE)	84.80 $\pm$ 0.46	44.90 $\pm$ 0.73	93.50 $\pm$ 0.54	84.99 $\pm$ 0.54
GAT (RWSE)	86.62 $\pm$ 0.53	48.58 $\pm$ 0.41	92.53 $\pm$ 0.65	85.02 $\pm$ 0.67
Gated-GCN	74.46 $\pm$ 0.54	43.00 $\pm$ 0.32	87.54 $\pm$ 1.22	77.31 $\pm$ 1.14
GCN	73.69 $\pm$ 0.74	48.70 $\pm$ 0.63	89.75 $\pm$ 0.52	83.64 $\pm$ 0.67
GCN (LapPE)	83.37 $\pm$ 0.55	44.35 $\pm$ 0.36	94.26 $\pm$ 0.49	84.95 $\pm$ 0.78
GCN (RWSE)	84.84 $\pm$ 0.55	46.40 $\pm$ 0.55	93.84 $\pm$ 0.48	85.11 $\pm$ 0.77
CO-GNN (Σ , Σ)	91.57 $\pm$ 0.32	51.28 $\pm$ 0.56	95.09 $\pm$ 1.18	83.36 $\pm$ 0.89
CO-GNN (μ , μ)	91.37 $\pm$ 0.35	<u>54.17$\pm$0.37</u>	97.31 $\pm$ 0.41	84.45 $\pm$ 1.17
SAGE	85.74 $\pm$ 0.67	53.63 $\pm$ 0.39	93.51 $\pm$ 0.57	82.43 $\pm$ 0.44
<i>Graph Transformers</i>				
Exphormer	89.03 $\pm$ 0.37	53.51 $\pm$ 0.46	90.74 $\pm$ 0.53	83.77 $\pm$ 0.78
NAGphormer	74.34 $\pm$ 0.77	51.26 $\pm$ 0.72	84.19 $\pm$ 0.66	78.32 $\pm$ 0.95
GOAT	71.59 $\pm$ 1.25	44.61 $\pm$ 0.50	81.09 $\pm$ 1.02	83.11 $\pm$ 1.04
GPS	82.00 $\pm$ 0.61	53.10 $\pm$ 0.42	90.63 $\pm$ 0.67	83.71 $\pm$ 0.48
GPSGCN+Performer (LapPE)	83.96 $\pm$ 0.53	48.20 $\pm$ 0.67	93.85 $\pm$ 0.41	84.72 $\pm$ 0.77
GPSGCN+Performer (RWSE)	84.72 $\pm$ 0.65	48.08 $\pm$ 0.85	92.88 $\pm$ 0.50	84.81 $\pm$ 0.86
GPSGCN+Transformer (LapPE)	OOM	OOM	91.82 $\pm$ 0.41	83.51 $\pm$ 0.93
GPSGCN+Transformer (RWSE)	OOM	OOM	91.17 $\pm$ 0.51	83.53 $\pm$ 1.06
GT	86.51 $\pm$ 0.73	51.17 $\pm$ 0.66	91.85 $\pm$ 0.76	83.23 $\pm$ 0.64
GT-sep	87.32 $\pm$ 0.39	52.18 $\pm$ 0.80	92.29 $\pm$ 0.47	82.52 $\pm$ 0.92
Polynormer	92.55$\pm$0.30	54.81$\pm$0.49	<u>97.46$\pm$0.36</u>	<u>85.91$\pm$0.74</u>
<i>Heterophily-Designated GNNs</i>				
CPGNN	63.96 $\pm$ 0.62	39.79 $\pm$ 0.77	52.03 $\pm$ 5.46	73.36 $\pm$ 1.01
FAGCN	65.22 $\pm$ 0.56	44.12 $\pm$ 0.30	88.17 $\pm$ 0.73	77.75 $\pm$ 1.05
FSGNN	79.92 $\pm$ 0.56	52.74 $\pm$ 0.83	90.08 $\pm$ 0.70	82.76 $\pm$ 0.61
GBK-GNN	74.57 $\pm$ 0.47	45.98 $\pm$ 0.71	90.85 $\pm$ 0.58	81.01 $\pm$ 0.67
GloGNN	59.63 $\pm$ 0.69	36.89 $\pm$ 0.14	51.08 $\pm$ 1.23	73.39 $\pm$ 1.17
GPR-GNN	64.85 $\pm$ 0.27	44.88 $\pm$ 0.34	86.24 $\pm$ 0.61	72.94 $\pm$ 0.97
H2GCN	60.11 $\pm$ 0.52	36.47 $\pm$ 0.23	89.71 $\pm$ 0.31	73.35 $\pm$ 1.01
JacobiConv	71.14 $\pm$ 0.42	43.55 $\pm$ 0.48	89.66 $\pm$ 0.40	68.66 $\pm$ 0.65
<i>Graph SSMs</i>				
GMN	87.69 $\pm$ 0.50	<i>54.07$\pm$0.31</i>	91.01 $\pm$ 0.23	84.52 $\pm$ 0.21
GPS + Mamba	83.10 $\pm$ 0.28	45.13 $\pm$ 0.97	89.93 $\pm$ 0.54	83.70 $\pm$ 1.05
GRAMA _{GCN}	88.61 $\pm$ 0.43	53.48 $\pm$ 0.62	95.27 $\pm$ 0.71	86.23$\pm$1.10
MP-SSM	90.91 $\pm$ 0.48	53.65 $\pm$ 0.71	95.33 $\pm$ 0.72	85.26 $\pm$ 0.93
<i>Spectral</i>				
Stable-ChebNet	92.03 $\pm$ 0.85	53.15 $\pm$ 0.21	95.71 $\pm$ 2.26	85.55 $\pm$ 3.35
Ours	<u>92.12$\pm$1.12</u>	53.39 $\pm$ 0.58	97.50$\pm$0.26	<i>85.57$\pm$0.61</i>

Type	Model	peptides-func (AP $\uparrow$)	peptides-struct (MAE $\downarrow$)
Trans.	SAN+LapPE	63.84 $\pm$ 1.21	0.2683 $\pm$ 0.0043
	TIGT	66.79 $\pm$ 0.74	0.2485 $\pm$ 0.0015
	Specformer	66.86 $\pm$ 0.64	0.2550 $\pm$ 0.0014
	Expformer	65.27 $\pm$ 0.43	0.2481 $\pm$ 0.0007
	G.MLPMixer	69.21 $\pm$ 0.54	0.2475 $\pm$ 0.0015
	Graph ViT	69.42 $\pm$ 0.75	0.2449 $\pm$ 0.0016
	GRIT	69.88 $\pm$ 0.82	0.2460 $\pm$ 0.0012
Rewir.	LASER	64.40 $\pm$ 0.10	0.3043 $\pm$ 0.0019
	DRew-GCN	69.96 $\pm$ 0.76	0.2781 $\pm$ 0.0028
	+PE	71.50 $\pm$ 0.44	0.2536 $\pm$ 0.0015
SS	Graph Mamba	67.39 $\pm$ 0.87	0.2478 $\pm$ 0.0016
	GMN	70.71 $\pm$ 0.83	0.2473 $\pm$ 0.0025
	MP-SSM	69.93 $\pm$ 0.52	0.2458 $\pm$ 0.0017
GNN	A-DGN	59.75 $\pm$ 0.44	0.2874 $\pm$ 0.0021
	ChebNet	69.61 $\pm$ 0.33	0.2627 $\pm$ 0.0033
	ChebNetII	68.19 $\pm$ 0.27	0.2618 $\pm$ 0.0058
	GCN	68.60 $\pm$ 0.50	0.2460 $\pm$ 0.0007
	GRAMA	70.93 $\pm$ 0.78	0.2436 $\pm$ 0.0022
	GRAND	57.89 $\pm$ 0.62	0.3418 $\pm$ 0.0015
	GraphCON	60.22 $\pm$ 0.68	0.2778 $\pm$ 0.0018
	PH-DGN	70.12 $\pm$ 0.45	0.2465 $\pm$ 0.0020
	SWAN	67.51 $\pm$ 0.39	0.2485 $\pm$ 0.0009
	PathNN	68.16 $\pm$ 0.26	0.2545 $\pm$ 0.0032
	CIN++	65.69 $\pm$ 1.17	0.2523 $\pm$ 0.0013
	S2GCN	72.75 $\pm$ 0.66	0.2467 $\pm$ 0.0019
	+PE	73.11 $\pm$ 0.66	0.2447 $\pm$ 0.0032
	Stable-ChebNet	70.32 $\pm$ 0.26	0.2542 $\pm$ 0.0030
	L2G-Net (ours)	<u>72.14</u> $\pm$ 0.24	0.2479 $\pm$ 0.0012
	+PE	<u>72.46</u> $\pm$ 0.25	0.2462 $\pm$ 0.0011

Table 13. Long-range benchmark results (Dwivedi et al., 2022b). AP is reported on peptides-func (higher is better), and MAE on peptides-struct (lower is better). Trans. stands for transformer, Rewir. for rewiring, and SS for state space.

Table 14. Average test accuracy with standard deviation for 4 random seeds on City-Networks (Liang et al., 2025).

Model	Paris	Shanghai	LA	London
MLP	25.5 _(0.4)	48.4 _(0.6)	24.1 _(0.5)	27.9 _(0.1)
ChebNet	54.1 _(0.2)	66.5 _(0.1)	61.4 _(0.4)	54.7 _(0.2)
GCN	53.2 _(0.3)	62.1 _(0.2)	58.3 _(0.3)	50.1 _(0.7)
SAGE	54.6 _(0.2)	68.3 _(0.5)	61.4 _(0.3)	55.4 _(0.2)
GAT	51.1 _(0.3)	68.0 _(0.5)	59.5 _(0.3)	52.0 _(0.3)
GCNII	51.3 _(0.2)	61.5 _(0.4)	56.0 _(0.3)	48.2 _(0.3)
DropEdge	48.2 _(0.2)	60.8 _(0.4)	55.5 _(0.3)	45.0 _(0.3)
GraphGPS	52.1 _(0.6)	63.0 _(0.5)	59.8 _(0.5)	OOM
Expformer	55.1 _(0.8)	70.2 _(0.4)	63.8 _(0.6)	49.5 _(0.4)
SGFormer	52.0 _(0.8)	64.1 _(0.3)	60.1 _(0.7)	48.3 _(0.3)
Ours	55.4 _(0.4)	69.8 _(0.5)	63.5 _(0.7)	55.6 _(0.6)